on the entries of A, we can assert that if ϵ is sufficiently small, $A + \delta A$ will have exactly one eigenvalue $\lambda + \delta\lambda$ that is close to λ. In Theorem 6.5.12 we will assume that all of these conditions hold.

Theorem 6.5.12 *Let $A \in \mathbb{C}^{n\times n}$ have n distinct eigenvalues. Let λ be an eigenvalue with associated right and left eigenvectors x and y, respectively, normalized so that $\|x\|_2 = \|y\|_2 = 1$. Let $s = y^*x$. (Then by Theorem 6.5.11 $s \neq 0$.) Define*

$$\kappa = \frac{1}{|s|} = \frac{1}{|y^*x|}.$$

Let δA be a small perturbation satisfying $\|\delta A\|_2 = \epsilon$, and let $\lambda+\delta\lambda$ be the eigenvalue of $A + \delta A$ that approximates λ. Then

$$|\delta\lambda| \leq \kappa\epsilon + O(\epsilon^2).$$

Thus κ is a condition number for the eigenvalue λ.[3]

Theorem 6.5.12 is actually valid for any simple eigenvalue, regardless of whether or not the matrix is semisimple.

Proof. From Theorem 6.5.3 we know that $|\delta\lambda| \leq \kappa_p(V)\epsilon$. This says that λ is not merely continuous in A, it is even Lipschitz continuous. This condition can be expressed briefly by the statement $|\delta\lambda| = O(\epsilon)$. It turns out that the same is true of the eigenvector as well: $A + \delta A$ has an eigenvector $x + \delta x$ associated with the eigenvalue $\lambda + \delta\lambda$, such that $\delta x = O(\epsilon)$. This depends on the fact that λ is a simple eigenvalue. For a proof see [81, p. 67]. Expanding the equation $(A + \delta A)(x + \delta x) = (\lambda + \delta\lambda)(x + \delta x)$ and using the fact that $Ax = \lambda x$, we find that

$$(\delta A)x + A(\delta x) + O(\epsilon^2) = (\delta\lambda)x + \lambda(\delta x) + O(\epsilon^2).$$

Left multiplying by y^* and using the equation $y^*A = \lambda y^*$, we obtain

$$y^*(\delta A)x + O(\epsilon^2) = (\delta\lambda)y^*x + O(\epsilon^2),$$

and therefore

$$\delta\lambda = \frac{y^*(\delta A)x}{y^*x} + O(\epsilon^2).$$

Taking absolute values and noting that $|y^*(\delta A)x| \leq \|y\|_2\|\delta A\|_2\|x\|_2 = \epsilon$, we are done. □

The relationship between κ and the overall condition number given by Theorem 6.5.3 is investigated in Exercise 6.5.29.

[3] λ is a *simple* eigenvalue; that is, its algebraic multiplicity is 1. Therefore x and y are chosen from one-dimensional eigenspaces. This and the fact that they have norm 1 guarantees that they are uniquely determined up to complex scalars of modulus 1. Hence s is determined up to a scalar of modulus 1, and κ is uniquely determined; that is, it is well defined.

Eigenvalue (λ_i)	Condition number (κ_i)
10, 1	4.5×10^3
9, 2	3.6×10^4
8, 3	1.3×10^5
7, 4	2.9×10^5
6, 5	4.3×10^5

Table 6.2 Condition numbers of eigenvalues of matrix A of Example 6.5.15

Exercise 6.5.13 Show that the condition number κ always satisfies $\kappa \geq 1$. □

Exercise 6.5.14 Let $A \in \mathbb{C}^{n\times n}$ be Hermitian ($A^* = A$), and let x be a right eigenvector associated with the eigenvalue λ.

(a) Show that x is also a left eigenvector of A.

(b) Show that the condition number of the eigenvalue λ is $\kappa = 1$.

This gives a second confirmation that the eigenvalues of a Hermitian matrix are perfectly conditioned. The results of this exercise also hold for normal matrices. See Exercise 6.5.29. □

Example 6.5.15 The 10×10 matrix

$$A = \begin{bmatrix} 10 & 10 & & & & & & & & \\ & 9 & 10 & & & & & & & \\ & & 8 & 10 & & & & & & \\ & & & \ddots & \ddots & & & & & \\ & & & & & 2 & 10 & & & \\ & & & & & & 1 & & & \end{bmatrix}$$

is a scaled-down version of an example from [81, p. 90]. The entries below the main diagonal and those above the superdiagonal are all zero. The eigenvalues are obviously $10, 9, 8, \ldots, 2, 1$. We calculated the right and left eigenvectors, and thereby obtained the condition numbers, which are shown in Table 6.2. The eigenvalues are listed in pairs because they possess a certain symmetry. Notice that the condition numbers are fairly large, but the extreme eigenvalues are not as ill conditioned as the ones in the middle of the spectrum. Let A_ϵ be the matrix that is the same as A, except that the (10,1) entry is ϵ instead of 0. This perturbation of norm ϵ should cause a perturbation in λ_i for which $\kappa_i\epsilon$ is a rough bound. Table 6.3 gives the eigenvalues of A_ϵ for $\epsilon = 10^{-6}$, as calculated by the QR algorithm. It also shows how much the eigenvalues deviate from those of A and gives the numbers $\kappa_i\epsilon$ for comparison. As you can see, the numbers $\kappa_i\epsilon$ give good order-of-magnitude estimates of the actual perturbations. Notice also that the extreme eigenvalues are still quite close to the original values, while those in the middle have wandered quite far. We can expect that

Eigenvalues of A	10, 1	9, 2	8, 3	7, 4	6, 5
Eigenvalues of A_ϵ	10.0027 .9973	8.9740 2.0260	8.0909 2.9091	6.6614 4.3386	6.4192 4.5808
Perturbations	.0027	.0260	.0909	.3386	.4192
$\kappa_i\epsilon$	.0045	.0361	.1326	.2931	.4281

Table 6.3 Comparison of eigenvalues of A and A_ϵ for $\epsilon = 10^{-6}$

if ϵ is made much larger, some of the eigenvalues will lose their identities completely. Indeed, for $\epsilon = 10^{-5}$ the eigenvalues of A_ϵ, as computed by the QR algorithm, are

$$\begin{array}{lll} 10.026 & 8.68 \pm .29i & 6.64 \pm .98i \\ .974 & 2.32 \pm .29i & 4.36 \pm .98i \end{array}$$

Only the two most extreme eigenvalues are recognizable. The others have collided in pairs to form complex conjugate eigenvalues. We conclude this example with two remarks. 1.) We also calculated the overall condition number $\kappa_2(V)$ given by Theorem 6.5.3, and found that it is about 2.5×10^6. This is clearly a gross overestimate of all of the individual condition numbers. 2.) The ill conditioning of the eigenvalues of A can be explained in terms of A's *departure from normality*. From Corollary 6.5.8 we know that the eigenvalues of a normal matrix are perfectly conditioned, and it is reasonable to expect that a matrix that is in some sense nearly normal would have well-conditioned eigenvalues. In Exercise 5.4.41 we found that a matrix that is upper triangular is normal if and only if it is a diagonal matrix. Our matrix A appears to be far from normal since it is upper triangular yet far from diagonal. Such matrices are said to have a high *departure from normality*. □

Exercise 6.5.16 MATLAB's command `condeig` is a variant of `eig` that computes the condition numbers given by Theorem 6.5.12. Type `help condeig` for information on how to use this command. Use `condeig` to check the numbers in Table 6.2. □

Sensitivity of Eigenvectors

In the proof of Theorem 6.5.12 we used the fact that if A has distinct eigenvalues, then the eigenvectors are Lipschitz continuous functions of the entries of A. This means that if x is an eigenvector of A, then any slightly perturbed matrix $A + \delta A$ will have an eigenvector $x + \delta x$ such that $\|\delta x\|_2 \approx \kappa_x \epsilon$. Here $\epsilon = \|\delta A\|_2$, and κ_x is a positive constant independent of δA. This is actually not hard to prove using notions from classical matrix theory and the theory of analytic functions; see Chapter 2 of [81]. Here we will take the existence of κ_x for granted and study its value, for κ_x is a condition number for the eigenvector x.

As we shall see, the computation of condition numbers for eigenvectors is more difficult than for eigenvalues, both in theory and in practice. We will proceed by degrees, and we will not attempt to cover every detail. We are guided by [66]. An

important strength of this approach is that it can be generalized to give condition numbers for invariant subspaces. See [66].

To begin with, let us assume that our matrix, now called T, is a block upper-triangular matrix

$$T = \begin{bmatrix} \lambda & w^T \\ 0 & \hat{T} \end{bmatrix}, \tag{6.5.17}$$

where the eigenvalues of $\hat{T}$ are all distinct from λ. Thus λ is a simple eigenvalue of T and has a one-dimensional eigenspace spanned by $e_1 = [1, 0, \cdots, 0]^T$. Suppose we perturb T by changing only the zero part. Thus we take

$$T + \delta T = \begin{bmatrix} \lambda & w^T \\ y & \hat{T} \end{bmatrix}, \tag{6.5.18}$$

where $\| y \|_2$ is small. Let us suppose that $\| \delta T \|_2 / \| T \|_2 = \| y \|_2 / \| T \|_2 = \epsilon \ll 1$. Then, if ϵ is small enough, $T + \delta T$ has an eigenvalue $\lambda + \delta\lambda$ near λ and an associated eigenvector $\begin{bmatrix} 1 \\ z \end{bmatrix}$ near e_1, which means that $\| z \|_2$ is small. Our task is to figure out just how small $\| z \|_2$ is. More precisely, we wish to find a condition number κ such that $\| z \|_2 \leq \kappa\epsilon$.

To this end, we write down the eigenvector equation that must be satisfied by this eigenpair of $T + \delta T$. We have

$$\begin{bmatrix} \lambda & w^T \\ y & \hat{T} \end{bmatrix} \begin{bmatrix} 1 \\ z \end{bmatrix} = \begin{bmatrix} 1 \\ z \end{bmatrix} (\lambda + \delta\lambda). \tag{6.5.19}$$

This can be written as two separate equations, the first of these is a scalar equation that tells us immediately that

$$\delta\lambda = w^T z. \tag{6.5.20}$$

If we use this in the second equation of (6.5.19), we obtain $y + \hat{T}z = z(\lambda + w^T z)$, which we rewrite as

$$(\hat{T} - \lambda I)z = -y + z(w^T z). \tag{6.5.21}$$

This is a nonlinear equation that we could solve for z in principle [66]. It is nonlinear because components of z are multiplied by components of z in the term $z(w^T z)$. The nonlinearity would make the equation more difficult to handle, but fortunately we can ignore it. Since $\| z \| = O(\epsilon)$, we have $\| z(w^T z) \| = O(\epsilon^2)$, so this term is insignificant. Therefore, since $\hat{T} - \lambda I$ is nonsingular, we can multiply (6.5.21) by $(\hat{T} - \lambda I)^{-1}$ to obtain

$$z = -(\hat{T} - \lambda I)^{-1} y + O(\epsilon^2)$$

and hence

$$\| z \|_2 \leq \| (\hat{T} - \lambda I)^{-1} \|_2 \| y \|_2 + O(\epsilon^2). \tag{6.5.22}$$

This result yields immediately the following lemma, which gives a condition number for the eigenvector e_1.

Lemma 6.5.23 *Let T be a block-triangular matrix of the form (6.5.17), where λ is distinct from the eigenvalues of $\hat{T}$. Let $T + \delta T$ be a perturbation of T of the special form (6.5.18), where $\|\delta T\|_2/\|T\|_2 = \epsilon$. Then, if ϵ is sufficiently small, $T + \delta T$ has an eigenvector $e_1 + \delta v$ such that*

$$\|\delta v\|_2 \leq \left(\|(\hat{T} - \lambda I)^{-1}\|_2 \|T\|_2\right)\epsilon + O(\epsilon^2).$$

Thus

$$\|(\hat{T} - \lambda I)^{-1}\|_2 \|T\|_2$$

is a condition number for the eigenvector e_1 with respect to perturbations of T of the special form given by (6.5.18).

Proof. Since $\|\delta v\|_2 = \|z\|_2$ and $\|\delta T\|_2 = \|y\|_2$, this result is an immediate consequence of (6.5.22). □

A couple of remarks are in order. 1.) This analysis also yields an expression for the perturbation of the eigenvalue λ, namely (6.5.20). The relationship between this expression and the condition number given by Theorem 6.5.12 is explored in Exercise 6.5.34. 2.) Equation (6.5.20) yields immediately $|\delta\lambda| \leq \|w\|_2\|z\|_2$, which suggests (correctly) that the norm of w affects the sensitivity of the eigenvalue λ. Although w is not mentioned in Lemma 6.5.23, it also affects the sensitivity of the eigenvector: it determines, in part, how small ϵ must be in order for Lemma 6.5.23 to be valid. For details see [66]. (In our analysis we obliterated w by replacing $z(w^T z)$ by the expression $O(\epsilon^2)$.)

Now consider a general matrix A with a simple eigenvalue λ and corresponding eigenvector x, normalized so that $\|x\|_2 = 1$. Let δA be a perturbation such that $\|\delta A\|_2/\|A\|_2 = \epsilon \ll 1$. Then $A + \delta A$ has an eigenvector $x + \delta x$ near x. We aim to derive a bound on $\|\delta x\|_2/\|x\|_2 = \|\delta x\|_2$.

Let V be any unitary matrix whose first column is x, and let $T = V^{-1}AV$. (For example, we could take the Schur decomposition of A. See Theorem 5.4.11.) Then T has the form

$$T = \begin{bmatrix} \lambda & w^T \\ 0 & \hat{T} \end{bmatrix},$$

where $\hat{T}$ has eigenvalues different from λ. Let $\delta T = V^{-1}\delta A V$. Then $A + \delta A = V(T + \delta T)V^{-1}$, and $\|\delta T\|_2/\|T\|_2 = \|\delta A\|_2/\|A\|_2 = \epsilon$. Writing

$$T + \delta T = \begin{bmatrix} \lambda + \delta t_{11} & w^T + \delta w^T \\ y & \hat{T} + \delta\hat{T} \end{bmatrix},$$

we have $|\delta t_{11}| \leq \|\delta T\|_2$, $\|\delta\hat{T}\|_2 \leq \|\delta T\|_2$, and $\|y\|_2 \leq \|\delta T\|_2$. If ϵ is small enough, then $\lambda + \delta t_{11}$ is distinct from all of the eigenvalues of $\hat{T} + \delta\hat{T}$, and we can apply Lemma 6.5.23 to the matrix

$$\tilde{T} = \begin{bmatrix} \lambda + \delta t_{11} & w^T + \delta w^T \\ 0 & \hat{T} + \delta\hat{T} \end{bmatrix}.$$

The perturbation $\delta\tilde{T} = \begin{bmatrix} 0 & 0 \\ y & 0 \end{bmatrix}$ gives $\tilde{T} + \delta\tilde{T} = T + \delta T$. We conclude that $T + \delta T$ has an eigenvector $e_1 + \delta v$ such that

$$\|\delta v\|_2 \leq \left(\|(\hat{T} + \delta\hat{T} - (\lambda + \delta t_{11})I)^{-1}\|_2 \|T\|_2\right) \frac{\|y\|_2}{\|T\|_2} + O(\epsilon^2).$$

Now let $\delta x = V\delta v$. Since also $x = Ve_1$ (first column of V), we have $x + \delta x = V(e_1 + \delta v)$. Since $e_1 + \delta v$ is an eigenvector of $T + \delta T$, $x + \delta x$ is an eigenvector of $A + \delta A$. The unitary transformation V preserves Euclidean norms, and $\|y\|_2 \leq \|\delta A\|_2$, so

$$\frac{\|\delta x\|_2}{\|x\|_2} \leq \left(\|(\hat{T} + \delta\hat{T} - (\lambda + \delta t_{11})I)^{-1}\|_2 \|A\|_2\right) \frac{\|\delta A\|_2}{\|A\|_2} + O(\epsilon^2)$$

if ϵ is sufficiently small. This result seems to say that we can use

$$\|(\hat{T} + \delta\hat{T} - (\lambda + \delta t_{11})I)^{-1}\|_2 \|A\|_2$$

as a condition number for the eigenvector x. Its obvious defect is that it depends upon δt_{11} and $\delta\hat{T}$, quantities that are unknown and unknowable. Since they are small quantities, we simply ignore them. Thus we take

$$\|(\hat{T} - \lambda I)^{-1}\|_2 \|A\|_2 \tag{6.5.24}$$

as the condition number of x.

Computation of the condition number (6.5.24) is no simple matter. Not only is a Schur-like decomposition $A = VTV^{-1}$ required, but we must also compute $\|(\hat{T} - \lambda I)^{-1}\|_2$. The latter task is the expensive one. In practice we can estimate this quantity just as one estimates $\|A^{-1}\|$, as discussed at the end of Section 2.2, and thereby obtain an estimate of the condition number. An estimate is usually good enough, since we are only interested in the magnitude of the condition number, not its exact value. A condition number estimator for this problem is included in LAPACK [1].

What general conclusions can we draw from the condition number (6.5.24)? If we use the Schur decomposition $A = VTV^{-1}$ (Theorem 5.4.11), $\hat{T}$ is upper triangular and has the eigenvalues of A, other than λ, on its main diagonal. Then $(\hat{T} - \lambda I)^{-1}$ is also upper triangular and has main diagonal entries of the form $(\lambda_i - \lambda)^{-1}$, where λ_i are the other eigenvalues of A. It follows easily that

$$\|(\hat{T} - \lambda I)^{-1}\|_2 \geq \max_i |\lambda_i - \lambda|^{-1} = \frac{1}{\min_i |\lambda_i - \lambda|}. \tag{6.5.25}$$

This number will be large if A has other eigenvalues that are extremely close to λ. We conclude that if λ is one of a cluster of (at least two) tightly packed eigenvalues, then its associated eigenvector will be ill conditioned.

Exercise 6.5.26 Show that if U is upper triangular, then $\|U\|_2 \geq \max_i |u_{ii}|$. Show that equality holds if U is diagonal. □

If A is normal, then the Schur matrix T is diagonal, and equality holds in (6.5.25). In this case we also have $\|A\|_2 = \max_i |\lambda_i|$, so the condition number of the eigenvector associated with simple eigenvalue λ_j is

$$\frac{\max_i |\lambda_i|}{\min_{i \neq j} |\lambda_i - \lambda_j|}. \tag{6.5.27}$$

We emphasize that this is for normal matrices only. In words, if A is normal, the condition number of the eigenvector associated with λ_j is inversely proportional to the distance from λ_j to the next nearest eigenvalue. Exercises 6.5.35 and 6.5.36 study the sensitivity of eigenvectors of normal matrices.

If A is not normal, then (6.5.27) is merely a lower bound on the condition number. It is often a good estimate; however, it can sometimes be a severe underestimate, because the gap in the inequality (6.5.25) can be very large when $\hat{T}$ is not normal. We conclude that it is possible for an eigenvector to be ill conditioned even though its associated eigenvalue is well separated from the other eigenvalues.

Additional Exercises

Exercise 6.5.28 In Exercise 5.3.19 we ascertained that the MATLAB test matrix `west0479` has an eigenvalue $\lambda \approx 17.548546093831+34.237822957500i$, and in Exercise 5.3.21 we computed a right eigenvector by one step of inverse iteration. By the same method we can get a left eigenvector. Therefore we can compute the condition number of λ given by Theorem 6.5.12. Here is some sample code, adapted from Exercise 5.3.19.

```
load west0479;
A = west0479;
n = 479;
lam = 17.548546093831 +34.237822957500i;
[L,U,P] = lu(A - lam*speye(n));
vright = ones(n,1);
vright = P*vright;
vright = L\vright;
vright = U\vright;
vright = vright/norm(vright);
vleft  = ones(n,1);
vleft = U'\vleft;
vleft = L'\vleft;
vleft = P'*vleft;
vleft = vleft/norm(vleft);
```

(a) This code segment computes a decomposition $A - \lambda I = P^T LU$, where P is a permutation matrix. (Type `help lu` for more information.) Show that

the instructions involving `vleft` compute a left eigenvector of A (i.e. a right eigenvector of A^*) by one step of inverse iteration.

(b) Add more code to compute $\|r\|_2$, where $r = Av - \lambda v$ is the residual for the right eigenvector, and compute an analogous residual for the left eigenvector. Both of the residuals should be tiny.

(c) Add more code to compute the condition number of λ given by Theorem 6.5.12. You should get $\kappa \approx 5 \times 10^5$. Thus λ is not very well conditioned.

(d) Using your computed values of κ and $\|r\|_2$, together with Theorems 6.5.1 and 6.5.12, decide how many of the digits in the expression $\lambda \approx 17.548546093831 + 34.237822957500i$ can be trusted.

□

Exercise 6.5.29 Let $A \in \mathbb{C}^{n\times n}$ have distinct eigenvalues $\lambda_1, \ldots, \lambda_n$ with associated right and left eigenvectors $v_1, \ldots, v_n$ and $w_1, \ldots, w_n$, respectively. By Theorem 6.5.11 we know that $w_i^* v_i \neq 0$ for all i, so we can assume without loss of generality that the vectors have been scaled so that $w_i^* v_i = 1$, $i = 1, \ldots, n$. This scaling does not determine the eigenvectors uniquely, but once v_i is chosen, then w_i^* is uniquely determined and vice versa. Let $V \in \mathbb{C}^{n\times n}$ be the matrix whose ith column is v_i, and let $W^* \in \mathbb{C}^{n\times n}$ be the matrix whose ith row is w_i^*.

(a) Show that $W^* = V^{-1}$.

(b) Show that the condition number κ_i associated with the ith eigenvalue is given by $\kappa_i = \|v_i\|_2 \|w_i\|_2$.

(c) Show that $\kappa_i \leq \kappa_2(V)$ for all i. Thus the overall condition number from Theorem 6.5.3 (with $p = 2$) always overestimates the individual condition numbers.

(d) Show that if A is normal, $\kappa_i = 1$ for all i.

□

Exercise 6.5.30 The Gerschgorin disk theorem is an interesting classical result that has been used extensively in the study of perturbations of eigenvalues. In this exercise you will prove the most basic version of the Gerschgorin disk theorem, and in Exercise 6.5.31 you will use the Gerschgorin theorem to derive a second proof of Theorem 6.5.3. We begin by defining the n Gerschgorin disks associated with a matrix $A \in \mathbb{C}^{n\times n}$. Associated with the ith row of A we have the ith off-diagonal row sum

$$s_i = \sum_{j\neq i} |a_{ij}|,$$

the sum of the absolute values of the entries of the ith row, omitting the main-diagonal entry. The ith *Gerschgorin disk* is the set of complex numbers z such that

$|z - a_{ii}| \leq s_i$. This is a closed circular disk in the complex plane, the set of complex numbers whose distance from a_{ii} does not exceed s_i. The matrix A has n Gerschgorin disks associated with it, one for each row. The Gerschgorin Disk Theorem states simply that the eigenvalues of A lie within the union of its n Gerschgorin disks. Prove this theorem by showing that each eigenvalue λ must lie within one of the Gerschgorin disks: Let λ be an eigenvalue of A, and let $x \neq 0$ be an associated eigenvector. Let the largest entry of x be the kth entry: $|x_k| = \max_j |x_j| > 0$. Show that

$$(\lambda - a_{ii})x_i = \sum_{j \neq i} a_{ij}x_j$$

for all i. Using this equation in the case $i = k$, taking absolute values, and the using the triangle inequality and the maximum property of $|x_k|$, deduce that λ lies within the kth Gerschgorin disk of A. □

Exercise 6.5.31 Apply the Gerschgorin disk theorem to the matrix $D + \delta D$ that appears in the proof of Theorem 6.5.3 to obtain a second proof of the Theorem 6.5.3 for the case $p = 1$. For more delicate applications of this useful theorem see [81]. □

Exercise 6.5.32 For small $\epsilon \geq 0$ and $n \geq 2$ consider the $n \times n$ matrix

$$J(\epsilon) = \begin{bmatrix} 0 & 1 & & & & \\ & 0 & 1 & & & \\ & & 0 & 1 & & \\ & & & \ddots & \ddots & \\ & & & & 0 & 1 \\ \epsilon & & & & & 0 \end{bmatrix}.$$

The entries that have been left blank are zeros. Notice that $J(0)$ is a Jordan block, a severely defective matrix. It has only the eigenvalue 0, repeated n times, with a one-dimensional eigenspace (cf. Exercise 5.2.14).

(a) Show that the characteristic polynomial of $J(\epsilon)$ is $\lambda^n - \epsilon$. Show that every eigenvalue of $J(\epsilon)$ satisfies $|\lambda| = \epsilon^{1/n}$. Thus they all lie on the circle of radius $\epsilon^{1/n}$ centered on 0. (In fact the eigenvalues are the nth roots of ϵ: $\lambda_k = \epsilon^{1/n}e^{2\pi ik/n}$, $k = 1, \ldots, n$.)

(b) Sketch the graph of the function $f_n(\epsilon) = \epsilon^{1/n}$ for small $\epsilon \geq 0$ for a few values of n, e.g. $n = 2, 3, 4$. Compare this with the graph of $g_\kappa(\epsilon) = \kappa\epsilon$ for a few values of κ. Show that $f_n'(\epsilon) \to \infty$ as $\epsilon \to 0$.

(c) Let $A = J(0)$, whose only eigenvalue is 0, and let $\delta A_\epsilon = J(\epsilon) - J(0)$. Let μ_ϵ be any eigenvalue of $A + \delta A_\epsilon$. Show that there is no real number κ such that $|\mu_\epsilon - 0| \leq \kappa ||\delta A||_p$ for all $\epsilon > 0$. Thus there is no finite condition number for the eigenvalues of A. (Remark: The eigenvalues of $J(\epsilon)$ are continuous in ϵ but not Lipschitz continuous.)

(d) Consider the special case $n = 17$ and $\epsilon = 10^{-17}$. Observe that $\|\delta A\|_p = 10^{-17}$ but the eigenvalues of $A + \delta A$ all satisfy $|\lambda| = 1/10$. Thus the tiny perturbation 10^{-17} causes the relatively large perturbation 1/10 in the eigenvalues.

□

Exercise 6.5.33 Let A be the defective matrix

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

Find left and right eigenvectors of A associated with the eigenvalue 0, and show that they satisfy $y^*x = 0$. Does this contradict Theorem 6.5.11? □

Exercise 6.5.34 Let T have the form (6.5.17).

(a) Show that T has a left eigenvector of the form $\begin{bmatrix} 1 & u^T \end{bmatrix}$, where $u^T(T - \lambda I) = -w^T$.

(b) Show that $\|u\|_2 < \kappa$, the condition number for λ given by Theorem 6.5.12.

(c) Using (6.5.20) and (6.5.21), show that $\delta\lambda = u^T y + O(\epsilon^2)$, and deduce that

$$|\delta\lambda| \leq \kappa \|\delta T\|_2 + O(\epsilon^2).$$

This gives a second proof of Theorem 6.5.12 for perturbations of this special kind.

□

Exercise 6.5.35 The matrix

$$A = \begin{bmatrix} \epsilon & 0 & 0 \\ 0 & -\epsilon & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

is Hermitian, hence normal. Its eigenvalues are obviously ϵ, $-\epsilon$, and 1. Suppose $0 < \epsilon \ll 1$, so that the eigenvalues $\pm\epsilon$ are poorly separated. Let

$$\delta A = \begin{bmatrix} -\epsilon & \epsilon & 0 \\ \epsilon & \epsilon & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Obviously $\|\delta A\| = O(\epsilon)$. Do the following using pencil and paper (or in your head).

(a) Find the eigenvectors of A associated with the eigenvalues $\pm\epsilon$.

(b) Find the eigenvalues of $A + \delta A$ and note that two of them are of order ϵ. Find the eigenvectors of $A + \delta A$ associated with these two eigenvalues, and notice that they are far from eigenvectors of A.

(c) Show that the two eigenvectors of part (a) span the same two-dimensional subspace as the eigenvectors of part (b). This is an invariant subspace of both A and $A + \delta A$. Although the individual eigenspaces are ill conditioned, the two-dimensional invariant subspace is well conditioned [66].

□

Exercise 6.5.36 Use MATLAB to investigate the sensitivity of the eigenvectors of the normal matrix

$$A = \begin{bmatrix} 1+\gamma & 0 \\ 0 & 1-\gamma \end{bmatrix}$$

for small γ. Build a perturbation δA by

```
deltaA = [ 0 eps ; eps 0 ]*randn
```

MATLAB's constant `eps` is the "machine epsilon," which is twice the unit roundoff u. Use MATLAB's `eig` command to compute the eigenvectors of $A + \delta A$ for the three values $\gamma = 10^{-9}$, 10^{-12}, and 10^{-15} (e.g. `gamma = 1e-9`). In each case compute the distance from the eigenvectors of $A + \delta A$ to the eigenvectors of A. Are your results generally consistent with (6.5.27)? □

6.6 METHODS FOR THE SYMMETRIC EIGENVALUE PROBLEM

For dense, nonsymmetric eigenvalue problems, the QR algorithm, following reduction to Hessenberg form, remains supreme. However, in the symmetric case, QR has serious competition. A rich variety of approaches has been developed and, toward the end of the 20th century, new faster and more accurate algorithms were discovered and fine tuned. These include Cuppen's divide-and-conquer algorithm, the differential quotient-difference algorithm, and the RRR algorithm for computing eigenvectors.

The accuracy issue is interesting and important. Results in Section 6.5 imply that the eigenvalues of a symmetric matrix are well conditioned in the following sense: If $A + \delta A$ is a slight perturbation of a symmetric matrix A, say $\|\delta A_2\|/\|A\|_2 = \epsilon$, then each eigenvalue μ of $A + \delta A$ is close to an eigenvalue λ of A, in the sense that

$$\frac{|\mu - \lambda|}{\|A\|_2} \leq \epsilon. \tag{6.6.1}$$

Since the QR algorithm is normwise backward stable, it computes the exact eigenvalues of a matrix $A + \delta A$ that is very near to A. The approximate eigenvalues computed by QR satisfy (6.6.1) with ϵ equal to a modest multiple of the computer's unit roundoff u. This is a good result, but one can imagine doing better. It would be better if we could get

$$\frac{|\mu - \lambda|}{|\lambda|} \leq \epsilon \tag{6.6.2}$$

for all nonzero λ. This is a more stringent bound, which says that all nonzero eigenvalues, whether large or tiny, are computed to full precision. In contrast, the

bound (6.6.1) implies only that the eigenvalues that are of about the same magnitude as $\| A \|_2$ are computed to full precision; tiny eigenvalues are not necessarily computed to high relative accuracy. For example, an eigenvalue with magnitude $10^{-6}\| A \|_2$ could have six digits less precision than the larger eigenvalues. This is in fact what happens when the QR algorithm is used. Some of the algorithms to be discussed in this section do better than that; they achieve the stronger bound (6.6.2).

Before discussing newer developments, we will describe briefly a very old method that continues to attract attention.

The Jacobi Method

The Jacobi method is one of the oldest numerical methods for the eigenvalue problem. It is older than matrix theory itself, dating back to Jacobi's 1846 paper [44]. Like most numerical methods, it was little used in the precomputer era. It enjoyed a brief renaissance in the 1950s, but in the 1960s it was supplanted as the method of choice by the QR algorithm. Later it attracted renewed interest because of its inherent parallelism. We will outline the method briefly. The first edition of this book [77] contained a much more detailed discussion of Jacobi's method, including parallel Jacobi schemes.

Let us begin by considering a 2×2 real symmetric matrix

$$A = \begin{bmatrix} a & b \\ b & d \end{bmatrix}.$$

It is not hard to show that there is a rotator

$$Q = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}$$

such that $Q^T A Q$ is diagonal:

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} a & b \\ b & d \end{bmatrix} \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}.$$

This solves the 2×2 symmetric eigenvalue problem. The details are worked out in Exercise 6.6.46.

Finding the eigenvalues of a 2×2 matrix is not an impressive feat. Now let us see what we can do with an $n \times n$ matrix. Of the algorithms that we will discuss in this section, Jacobi's method is the only one that does not begin by reducing the matrix to tridiagonal form. Instead it moves the matrix directly toward diagonal form by setting off-diagonal entries to zero, one after the other. It is now clear that we can set any off diagonal entry a_{ij} to zero by an appropriate plane rotator: Just apply in the (i, j) plane the rotator that diagonalizes

$$\begin{bmatrix} a_{ii} & a_{ij} \\ a_{ij} & a_{jj} \end{bmatrix}.$$

Rotators that accomplish this task are called *Jacobi rotators*.

The *classical Jacobi method* searches the matrix for the largest off-diagonal entry and sets it to zero. Then it searches again for the largest remaining off-diagonal entry and sets that to zero, and so on. Since an entry that has been set to zero can be made nonzero again by a later rotator, we cannot expect to reach diagonal form in finitely many steps (and thereby violate Abel's Theorem). At best we can hope that the infinite sequence of Jacobi iterates will converge to diagonal form. Exercise 6.6.52 shows that this hope is realized. Indeed the convergence becomes quite swift, once the matrix is sufficiently close to diagonal form. This method was employed by Jacobi [44] in the solution of an eigenvalue problem that arose in a study of the perturbations of planetary orbits. The system was of order seven because there were then seven known planets. In the paper Jacobi stressed that the computations (by hand!) are quite easy. That was easy for him to say, for the actual computations were done by a student.

The classical Jacobi procedure is quite appropriate for hand calculation but inefficient for computer implementation. For a human being working on a small matrix it is a simple matter to identify the largest off-diagonal entry. the hard part is the arithmetic. However, for a computer working on a larger matrix, the arithmetic is easy; the search for the largest entry is the expensive part. See Exercise 6.6.51. Because the search is so time consuming, a class of variants of Jacobi's procedure, called *cyclic Jacobi methods*, was introduced. A cyclic Jacobi method sweeps through the matrix, setting entries to zero in some prespecified order and paying no attention to the magnitudes of the entries. In each complete *sweep*, each off-diagonal entry is set to zero once. For example, we could perform a *sweep by columns*, which would create zeros in the order

$$(2,1),\ (3,1),\ldots,(n,1),\ (3,2),\ (4,2),\ldots,(n,2),\ldots,(n,n-1). \qquad (6.6.3)$$

Alternatively one could sweep by rows or by diagonals, for example. We will refer to the method defined by the ordering (6.6.3) as the *special cyclic Jacobi method.* One can show that repeated special cyclic Jacobi sweeps result in convergence to diagonal form [23]. The analysis is more difficult than it is for classical Jacobi.

We remarked earlier that the classical Jacobi method converges quite swiftly, once the off-diagonal entries become fairly small. The same is true of cyclic Jacobi methods. In fact the convergence is quadratic, in the sense that if the off-diagonal entries are all $O(\epsilon)$ after a given sweep, they will be $O(\epsilon^2)$ after the next sweep. Exercise 6.6.53 shows clearly, if not rigorously, why this is so.

The accuracy of Jacobi's method depends upon the stopping criterion. Demmel and Veselic [14] have shown that if the algorithm is run until each a_{ij} is tiny relative to both a_{ii} and a_{jj}, then all eigenvalues are computed to high relative accuracy. That is, (6.6.2) is achieved. Thus the Jacobi method is more accurate than the QR algorithm.

If eigenvectors are wanted, they can be obtained by accumulating the product of the Jacobi rotators. A complete set of eigenvectors, orthonormal to working precision, is produced.

For either task, just computing eigenvalues or computing both eigenvalues and eigenvectors, the Jacobi method is several times slower than a reduction to tridiagonal form, followed by the QR algorithm.

Algorithms for Tridiagonal Matrices

In Section 5.5 we saw how to reduce any real, symmetric matrix to tridiagonal form

$$A = \begin{bmatrix} \alpha_1 & \beta_1 & & & & & \\ \beta_1 & \alpha_2 & \beta_2 & & & & \\ & \beta_2 & \alpha_3 & \ddots & & & \\ & & \ddots & \ddots & \beta_{n-2} & & \\ & & & \beta_{n-2} & \alpha_{n-1} & \beta_{n-1} & \\ & & & & \beta_{n-1} & \alpha_n & \end{bmatrix} \tag{6.6.4}$$

by an orthogonal similarity transformation. Without loss of generality, we will assume that all of the β_j are positive. From there we can reduce the tridiagonal matrix to diagonal form by the QR algorithm (Sections 5.6 and 5.7), for example. However, there are several alternatives for this second step.

Factored Forms of Tridiagonal Matrices

If all of the leading principal submatrices of A are nonsingular (for example, if A is positive definite), then by Theorem 1.7.30, A has a decomposition $A = LDL^T$, where L is unit lower triangular and D is diagonal. It is a routine matter to compute this decomposition by Gaussian elimination. When A is tridiagonal, the computation is especially simple and inexpensive. We will derive it here in a few lines, even though it is a special case of an algorithm that we already discussed in Chapter 1. Since A is tridiagonal, L must be bidiagonal, so the equation $A = LDL^T$ can be written more explicitly as

$$\begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \end{bmatrix}$$

$$= \begin{bmatrix} 1 & & & \\ l_1 & 1 & & \\ & \ddots & \ddots & \\ & & l_{n-1} & 1 \end{bmatrix} \begin{bmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{bmatrix} \begin{bmatrix} 1 & l_1 & & \\ & 1 & \ddots & \\ & & \ddots & l_{n-1} \\ & & & 1 \end{bmatrix}. \tag{6.6.5}$$

Exercise 6.6.6

(a) Multiply the matrices together on the right hand side of (6.6.5) and equate the entries on the right and left sides to obtain a system of $2n - 1$ equations.

(b) Verify that the entries of L and D are obtained from those of A by the following algorithm.

$$\begin{array}{l} d_1 \leftarrow \alpha_1 \\ \text{for } j = 1, \ldots, n-1 \\ \quad \left[\begin{array}{l} l_j \leftarrow \beta_j / d_j \\ d_{j+1} \leftarrow \alpha_{j+1} - d_j l_j^2 \end{array} \right. \end{array} \tag{6.6.7}$$

(c) About how many flops does this algorithm take?

□

The matrix A is determined by the $2n-1$ parameters $\alpha_1, \ldots, \alpha_n$ and $\beta_1, \ldots, \beta_{n-1}$. If we perform the decomposition $A = LDL^T$, we obtain an equal number of parameters $l_1, \ldots, l_{n-1}$ and $d_1, \ldots, d_n$, which encode the same information. Exercise 6.6.6 shows that the second parametrization can be obtained from the first in about $4n$ flops. Conversely, we can retrieve $\alpha_1, \ldots, \alpha_n$ and $\beta_1, \ldots, \beta_{n-1}$ from $l_1, \ldots, l_{n-1}$ and $d_1, \ldots, d_n$ with comparable speed. In principle either parametrization is an equally good representation of A.

In practice, however, there is a difference. The parameters $\alpha_1, \ldots, \alpha_n$ and $\beta_1, \ldots, \beta_{n-1}$ determine all eigenvalues of A to high absolute accuracy, but not to high relative accuracy. That is, if A has some tiny eigenvalues, then small relative perturbations of the parameters can cause large relative changes in those eigenvalues. The parameters $l_1, \ldots, l_{n-1}$ and $d_1, \ldots, d_n$ are better behaved in that regard. In the positive definite case, they determine all of the eigenvalues of LDL^T, even the tiniest ones, to high relative accuracy [55, 56]. This property nearly always holds in the non-positive case as well, although it is not guaranteed. Any factorization LDL^T whose parameters determine the eigenvalues to high relative accuracy is called a *relatively robust representation* (RRR) of the matrix.

These observations have the following consequence. If we want to produce highly accurate algorithms, we should develop algorithms that operate on the factored form of the matrix. The algorithms presented below do just that. Once a factored form is obtained, the tridiagonal matrix is never again explicitly formed.

As we have seen, a common operation in eigenvalue computations is to shift the origin, that is, to replace A by $A - \rho I$, where ρ is a shift. As a first task, let us see how we can effect a shift on a factored form of A. Say we have A in the factored form LDL^T. Our task is to compute unit lower-bidiagonal $\hat{L}$ and diagonal $\hat{D}$ such that $LDL^T - \rho I = \hat{L}\hat{D}\hat{L}^T$. By comparing entries in this equation, one easily checks that $\hat{L}$ and $\hat{D}$ can be computed by the following algorithm.

$$\begin{array}{l} \hat{d}_1 \leftarrow d_1 - \rho \\ \text{for } j = 1, \ldots, n-1 \\ \quad \left[\begin{array}{l} \hat{l}_j \leftarrow l_j d_j / \hat{d}_j \\ \hat{d}_{j+1} \leftarrow d_{j+1} + d_j l_j^2 - \hat{d}_j \hat{l}_j^2 - \rho \end{array} \right. \end{array} \tag{6.6.8}$$

Exercise 6.6.9 Show that if $LDL^T - \rho I = \hat{L}\hat{D}\hat{L}^T$, then

(a) $d_j + d_{j-1} l_{j-1}^2 - \rho = \hat{d}_j + \hat{d}_{j-1} \hat{l}_{j-1}^2$ for $j = 1, \ldots, n$, where $l_0 = \hat{l}_0 = d_0 = \hat{d}_0 = 0$.

(b) $l_j d_j = \hat{l}_j \hat{d}_j$ for $j = 1, \ldots, n-1$.

(c) Verify that (6.6.8) computes $\hat{L}$ and $\hat{D}$.

□

A stabler version of the algorithm is obtained by introducing the auxiliary quantities

$$s_j = d_{j-1} l_{j-1}^2 - \hat{d}_{j-1} \hat{l}_{j-1}^2 - \rho = \hat{d}_j - d_j, \tag{6.6.10}$$

with $s_1 = -\rho$. One easily checks that

$$s_{j+1} = \hat{l}_j l_j s_j - \rho. \tag{6.6.11}$$

This gives the so-called *differential* form of the algorithm.

Given L, D, and ρ, this algorithm produces $\hat{L}$ and $\hat{D}$ such that $\hat{L}\hat{D}\hat{L}^T = LDL^T - \rho$.

$$\begin{array}{l} s_1 \leftarrow -\rho \\ \text{for } j = 1, \ldots, n-1 \\ \quad \left[\begin{array}{l} \hat{d}_j \leftarrow d_j + s_j \\ \hat{l}_j \leftarrow l_j d_j / \hat{d}_j \\ s_{j+1} \leftarrow \hat{l}_j l_j s_j - \rho \end{array} \right. \\ \hat{d}_n \leftarrow d_n + s_n \end{array} \tag{6.6.12}$$

Obviously the flop count for (6.6.12) is $O(n)$.

Exercise 6.6.13

(a) Show that if s_j are defined by (6.6.10) with $s_1 = -\rho$, then the recursion (6.6.11) is valid. (For example, start with $s_{j+1} = d_j l_j l_j - \hat{d}_j \hat{l}_j \hat{l}_j - \rho$, and make a couple of substitutions using the equation $l_j d_j = \hat{l}_j \hat{d}_j$.)

(b) Verify that (6.6.12) performs the same function as (6.6.8).

□

A useful alternative to the LDL^T decomposition is obtained by disturbing the symmetry of A in a constructive way. It is easy to show that the symmetric, tridiagonal matrix A of (6.6.4) can be transformed to the form

$$\tilde{A} = \Delta^{-1} A \Delta = \begin{bmatrix} \alpha_1 & 1 & & & & \\ \beta_1^2 & \alpha_2 & 1 & & & \\ & \beta_2^2 & \alpha_3 & \ddots & & \\ & & \ddots & \ddots & 1 & \\ & & & \beta_{n-2}^2 & \alpha_{n-1} & 1 \\ & & & & \beta_{n-1}^2 & \alpha_n \end{bmatrix}, \tag{6.6.14}$$

where $\Delta = \operatorname{diag}\{\delta_1, \ldots, \delta_n\}$ is a diagonal transforming matrix (Exercise 6.6.54). Conversely, any matrix of the form

$$\tilde{A} = \begin{bmatrix} \alpha_1 & 1 & & & & \\ \gamma_1 & \alpha_2 & 1 & & & \\ & \gamma_2 & \alpha_3 & \ddots & & \\ & & \ddots & \ddots & 1 & \\ & & & \gamma_{n-2} & \alpha_{n-1} & 1 \\ & & & & \gamma_{n-1} & \alpha_n \end{bmatrix} \tag{6.6.15}$$

with all $\gamma_j > 0$ is diagonally similar to a symmetric matrix of the form (6.6.4) with $\beta_j = \sqrt{\gamma_j}$, $j = 1, \ldots, n-1$. It might seem like heresy to destroy the symmetry of a matrix, but it turns out that the form (6.6.15) is often useful.

If the matrix $\tilde{A}$ in (6.6.15) has all leading principal submatrices nonsingular, then it has an LU decomposition. In the context of eigenvalue computations, the symbol R is often used instead of U, so we will speak of an LR decomposition and write $\tilde{A} = \tilde{L}\tilde{R}$. The factors have the particularly simple form

$$\begin{bmatrix} 1 & & & & & \\ \tilde{l}_1 & 1 & & & & \\ & \tilde{l}_2 & 1 & & & \\ & & \ddots & \ddots & & \\ & & & \tilde{l}_{n-2} & 1 & \\ & & & & \tilde{l}_{n-1} & 1 \end{bmatrix} \begin{bmatrix} \tilde{r}_1 & 1 & & & & \\ & \tilde{r}_2 & 1 & & & \\ & & \tilde{r}_3 & \ddots & & \\ & & & \ddots & 1 & \\ & & & & \tilde{r}_{n-1} & 1 \\ & & & & & \tilde{r}_n \end{bmatrix}. \tag{6.6.16}$$

The $2n-1$ parameters $\tilde{l}_1, \ldots, \tilde{l}_{n-1}$ and $\tilde{r}_1, \ldots, \tilde{r}_n$ encode the same information as $\alpha_1, \ldots, \alpha_n$ and $\beta_1, \ldots, \beta_{n-1}$, in principle. However, just as in the case of the LDL^T decomposition, the entries of $\tilde{L}$ and $\tilde{R}$ usually determine the eigenvalues to high relative accuracy, whereas those of $\tilde{A}$ do not. Thus it makes sense to develop algorithms that operate directly on the parameters $\tilde{l}_1, \ldots, \tilde{l}_{n-1}$ and $\tilde{r}_1, \ldots, \tilde{r}_n$, never forming the tridiagonal matrix $\tilde{A}$ explicitly.

Obviously the parameters $\tilde{l}_1, \ldots, \tilde{l}_{n-1}$ and $\tilde{r}_1, \ldots, \tilde{r}_n$ must be closely related to the parameters $l_1, \ldots, l_{n-1}$ and $d_1, \ldots, d_n$ of the decomposition $A = LDL^T$. Indeed, it turns out that $\tilde{r}_j = d_j$ and $\tilde{l}_j = l_j \delta_{j-1}/\delta_j$, where the δ_j are the entries of the diagonal matrix Δ of (6.6.14).

Exercise 6.6.17 Let $A = LDL^T$ as in (6.6.5), $\tilde{A} = \Delta^{-1} A \Delta$ as in (6.6.14), and $\tilde{A} = \tilde{L}\tilde{R}$ as in (6.6.16).

(a) Show that $\tilde{L} = \Delta^{-1} L \Delta$ and $\tilde{R} = \Delta^{-1} D L^T \Delta$.

(b) Deduce that $\tilde{l}_j = l_j \delta_{j-1}/\delta_j$ for $j = 1, \ldots, n-1$ and $\tilde{r}_j = d_j$ for $j = 1, \ldots, n$.

□

We noted above how to effect a shift of origin on an LDL^T decomposition without explicitly forming the product A ((6.6.12) and Exercise 6.6.13). Naturally we can do the same thing with an LR decomposition. Suppose we have $\tilde{A} = \tilde{L}\tilde{R}$ in factored form, and we wish to compute $\hat{L}\hat{R}$ such that $\tilde{A} - \rho I = \hat{L}\hat{R}$ without explicitly forming $\tilde{A}$. One easily shows (Exercise 6.6.55) that the following algorithm produces $\hat{L}$ and $\hat{R}$ from $\tilde{L}$, $\tilde{R}$, and ρ.

Given $\tilde{L}$, $\tilde{R}$, and ρ, this algorithm produces $\hat{L}$ and $\hat{R}$ such that $\hat{L}\hat{R} = \tilde{L}\tilde{R} - \rho$.

$$
\begin{array}{l}
t_1 \leftarrow -\rho \\
\text{for } j = 1, \ldots, n-1 \\
\quad \left[\begin{array}{l}
\hat{r}_j \leftarrow \tilde{r}_j + t_j \\
q \leftarrow \tilde{l}_j / \hat{r}_j \\
\hat{l}_j \leftarrow \tilde{r}_j q \\
t_{j+1} \leftarrow t_j q - \rho
\end{array} \right. \\
\hat{r}_n = r_n + t_n
\end{array}
\tag{6.6.18}
$$

The Slicing or Bisection Method

The slicing method, also known as the bisection or Sturm sequence method, is based on the notions of congruence and inertia. We begin by noting that for any symmetric matrix $A \in \mathbb{R}^{n\times n}$ and any $S \in \mathbb{R}^{n\times n}$, SAS^T is also symmetric. Now let A and B be any two symmetric matrices in $\mathbb{R}^{n\times n}$. We say that B is *congruent* to A if there exists a nonsingular matrix $S \in \mathbb{R}^{n\times n}$ such that $B = SAS^T$. It is important that S is nonsingular.

Exercise 6.6.19 Show that (a) A is congruent to A, (b) if B is congruent to A, then A is congruent to B, (c) if A is congruent to B and B is congruent to C, then A is congruent to C. In other words, congruence is an equivalence relation. □

The eigenvalues of a symmetric $A \in \mathbb{R}^{n\times n}$ are all real. Let $\nu(A)$, $\zeta(A)$, and $\pi(A)$ denote the number of negative, zero, and positive eigenvalues of A, respectively. The ordered triple $(\nu(A), \zeta(A), \pi(A))$ is called the *inertia* of A. The key result of our development is Sylvester's law of inertia, which states that congruent matrices have the same inertia.

Theorem 6.6.20 *(Sylvester's Law of Inertia) Let $A, B \in \mathbb{R}^{n\times n}$ be symmetric, and suppose B is congruent to A. Then $\nu(A) = \nu(B)$, $\zeta(A) = \zeta(B)$, and $\pi(A) = \pi(B)$.*

For a proof see Exercise 6.6.56. We will use Sylvester's law to help us slice the spectrum into subsets. Let $\lambda_1, \ldots, \lambda_n$ be the eigenvalues of A, ordered so that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. Suppose that for any given $\rho \in \mathbb{R}$ we can determine the inertia of $A - \rho I$. The number $\pi(A - \rho I)$ equals the number of eigenvalues of A that are greater than ρ. If $\pi(A - \rho I) = i$, with $0 < i < n$, then

$$\lambda_n \leq \cdots \leq \lambda_{i+1} \leq \rho < \lambda_i \leq \cdots \leq \lambda_1.$$

This splits the spectrum into two subsets. We will see that by repeatedly slicing the spectrum with systematically chosen values of ρ, we can determine all the eigenvalues of A with great precision.

Exercise 6.6.21 Suppose you have a subroutine that can calculate $\pi(A - \rho I)$ for any value of ρ quickly. Devise an algorithm that uses this subroutine repeatedly to calculate all of the eigenvalues of A in some specified interval with error less than some specified tolerance $\epsilon > 0$. □

We now observe that if A is tridiagonal, we do in fact have inexpensive ways to calculate $\pi(A - \rho I)$ for any ρ. We need only calculate the decomposition

$$A - \rho I = L_\rho D_\rho L_\rho^T, \tag{6.6.22}$$

with L_ρ unit lower triangular and D_ρ diagonal, in $O(n)$ flops (Exercise 6.6.6). This equation is a congruence between $A - \rho I$ and D_ρ, so $\pi(A - \rho I) = \pi(D_\rho)$. Since D_ρ is diagonal, we find $\pi(D_\rho)$ by counting the number of positive elements on its main diagonal.

If we wish, we can factor A once into LDL^T and then use algorithm (6.6.12) to obtain the desired shifted factorizations without ever assembling any of the matrices $A - \rho I$ explicitly. Alternatively, we can transform A to the unsymmetric form $\tilde{A}$ and compute the decomposition $\tilde{L}\tilde{R}$. By Exercise 6.6.17, the main diagonal entries of $\tilde{R}$ are the same as those of D, so this decomposition also reveals the inertia of A. To obtain decompositions of shifted matrices, we use algorithm (6.6.18).

In either (6.6.12) or (6.6.18) it can happen that $\hat{d}_j = 0$ (resp $\hat{r}_j = 0$) for some j, which will lead to a division by zero. This poses no problem; for example, one can simply replace the zero by an extremely tiny number $\hat{\epsilon}$ (e.g. 10^{-307}) and continue. This has the same effect as replacing $\hat{d}_j$ by $\hat{d}_j + \hat{\epsilon}$, which perturbs the spectrum by a negligible amount.

Now that we know how to calculate $\pi(A - \rho I)$ for any value of ρ, let us see how we can systematically determine the eigenvalues of A. A straightforward bisection approach works very well. Suppose we wish to find all eigenvalues in the interval $(a, b]$. We begin by calculating $\pi(A - aI)$ and $\pi(A - bI)$ to find out how many eigenvalues lie in the interval. If $\pi(A - aI) = i$ and $\pi(A - bI) = j$, then

$$a < \lambda_i \leq \cdots \leq \lambda_{j+1} \leq b,$$

so there are $i - j$ eigenvalues in the interval. Now let $\rho = (a + b)/2$, the midpoint of the interval, and calculate $\pi(A - \rho I)$. From this we can deduce how many eigenvalues lie in each of the intervals $(a, \rho]$ and $(\rho, b]$. More precise information can be obtained by bisecting each of these subintervals, and so on. Any interval that is found to contain no eigenvalues can be removed from further consideration. An interval that contains a single eigenvalue can be bisected repeatedly until the eigenvalue has been located with sufficient precision. If we know that $\lambda_k \in (\rho_1, \rho_2]$, where $\rho_2 - \rho_1 < 2\epsilon$, then the approximation $\lambda_k \approx (\rho_1 + \rho_2)/2$ is in error by less than ϵ.

The LR and Quotient-Difference Algorithms

Another approach to computing the eigenvalues of a symmetric, tridiagonal matrix is the *quotient-difference* (qd) algorithm. We begin by introducing the LR algorithm, which will serve as a stepping stone to the qd algorithm. To this end we transform A to the nonsymmetric form $\tilde{A} = \Delta^{-1} A \Delta$ (6.6.15). For notational simplicity, we will now leave off the tildes and work with the nonsymmetric matrix

$$A = \begin{bmatrix} \alpha_1 & 1 & & & & \\ \gamma_1 & \alpha_2 & 1 & & & \\ & \gamma_2 & \alpha_3 & \ddots & & \\ & & \ddots & \ddots & 1 & \\ & & & \gamma_{n-2} & \alpha_{n-1} & 1 \\ & & & & \gamma_{n-1} & \alpha_n \end{bmatrix}. \tag{6.6.23}$$

If A has all leading principal submatrices nonsingular, then it has an LR decomposition.

$$\begin{bmatrix} 1 & & & & & \\ l_1 & 1 & & & & \\ & l_2 & 1 & & & \\ & & \ddots & \ddots & & \\ & & & l_{n-2} & 1 & \\ & & & & l_{n-1} & 1 \end{bmatrix} \begin{bmatrix} r_1 & 1 & & & & \\ & r_2 & 1 & & & \\ & & r_3 & \ddots & & \\ & & & \ddots & 1 & \\ & & & & r_{n-1} & 1 \\ & & & & & r_n \end{bmatrix}. \tag{6.6.24}$$

If we reverse the order of these factors and multiply them back together, we obtain

$$\hat{A} = \begin{bmatrix} r_1 + l_1 & 1 & & & & \\ r_2 l_1 & r_2 + l_2 & 1 & & & \\ & r_3 l_2 & r_3 + l_3 & \ddots & & \\ & & \ddots & \ddots & 1 & \\ & & & r_{n-1} l_{n-2} & r_{n-1} + l_{n-1} & 1 \\ & & & & r_n l_{n-1} & r_n \end{bmatrix}, \tag{6.6.25}$$

a matrix of the same form as A. The transformation from A to $\hat{A}$, which is easily seen to be a similarity transformation, is one step of the *LR algorithm*, which is analogous to the QR algorithm. If we iterate the process, the subdiagonal entries will converge gradually to zero, revealing the eigenvalues on the main diagonal. Just as for the QR algorithm, we can accelerate the convergence by using shifts of origin.[4]

[4]The LR algorithm can be applied to general matrices or upper Hessenberg matrices, not just tridiagonal ones. The convergence theory and implementation details are similar to those of the QR algorithm, but there are a few important differences. For one thing, if an unlucky shift ρ is chosen, $A - \rho I$ may fail

A sequence of LR iterations produces a sequence of similar matrices $A = A_0, A_1, A_2, A_3, \ldots$. Each matrix is obtained from the previous one via an LR decomposition. We arrive at the quotient-difference (qd) algorithm by shifting our attention from the sequence of matrices (A_j) to the sequence of intervening LR decompositions. The qd algorithm jumps from one LR decomposition to the next, bypassing the intermediate tridiagonal matrix. The advantage of this change of viewpoint is that it leads to more accurate algorithms since, as we have already mentioned, the spectrum of the matrix is more accurately encoded by the factored form.

It is not hard to figure out how to jump from one LR decomposition to the next. Suppose we have a decomposition LR of some iterate A. To complete an LR iteration in the conventional way, we would form $\hat{A} = RL$. To start the next iteration, we would subtract a shift ρ from $\hat{A}$ and perform a new decomposition $\hat{A} - \rho I = \hat{L}\hat{R}$. Our task now is to obtain the $\hat{L}$ and $\hat{R}$ directly from L and R (and ρ) without ever forming the intermediate matrix $\hat{A}$. Equating the matrix entries in the equation $RL - \rho I = \hat{L}\hat{R}$, we easily determine (Exercise 6.6.58) that the parameters $\hat{l}_j$ and $\hat{r}_j$ can be obtained from the l_j and r_j by the simple algorithm

$$\begin{array}{l} \hat{l}_0 \leftarrow 0 \\ \text{for } j = 1 \ldots, n-1 \\ \quad \left[\begin{array}{l} \hat{r}_j \leftarrow r_j + l_j - \rho - \hat{l}_{j-1} \\ \hat{l}_j \leftarrow l_j r_{j+1} / \hat{r}_j \end{array} \right. \\ \hat{r}_n \leftarrow r_n - \rho - \hat{l}_{n-1} \end{array} \tag{6.6.26}$$

This is one iteration of the *shifted quotient-difference* (qds) algorithm. If it is applied iteratively, it effects a sequence of iterations of the LR algorithm without ever forming the "intermediate" tridiagonal matrices. As $l_{n-1} \to 0$, r_n converges to an eigenvalue.

There is a slight difference in the way the shifts are handled. At each iteration we subtract a shift ρ, and we do not bother to restore the shift at the end of the iteration. Instead, we keep track of the accumulated shift. At each iteration we add the new shift ρ to the accumulated shift, which we can call $\hat{\rho}$. As each eigenvalue emerges, we must realize that it is the eigenvalue of a shifted matrix. To get the correct eigenvalue of the original matrix, we have to add on the accumulated shift $\hat{\rho}$.

To obtain a numerically superior algorithm, we have to rearrange (6.6.26). We introduce auxiliary quantities $s_j = r_j - \rho - \hat{l}_{j-1}$, $j = 1, \ldots, n$, where $\hat{l}_0 = 0$. Then we can write (6.6.26) as

to have an LR decomposition. Even worse, if ρ is close to an "unlucky" shift, the LR decomposition will be unstable, introducing unacceptable roundoff errors into the computation. For upper Hessenberg matrices there are bulge-chasing implicit single and double LR steps analogous to the implicit QR steps. Pivoting can be introduced to mitigate the instability. However, if the matrix is tridiagonal to begin with, the pivoting will ruin the tridiagonal form.

Shifted differential quotient-difference (dqds) iteration

$$
\begin{array}{l}
s_1 \leftarrow r_1 - \rho \\
\text{for } j = 1, \ldots, n-1 \\
\quad \left[\begin{array}{l}
\hat{r}_j \leftarrow s_j + l_j \\
q \leftarrow r_{j+1}/\hat{r}_j \\
\hat{l}_j \leftarrow l_j q \\
s_{j+1} \leftarrow s_j q - \rho
\end{array} \right. \\
\hat{r}_n \leftarrow s_n
\end{array}
\tag{6.6.27}
$$

The details are worked out in Exercise 6.6.59.

In the positive-definite case, the algorithm dqds (6.6.27) is extremely stable. One easily shows that in that case all of the quantities r_j and l_j are positive (Exercise 6.6.60). Looking at (6.6.27) in the case $\rho = 0$, we see that the s_j are also necessarily positive. It follows that there are no subtractions in (6.6.27). In the one addition operation, two positive quantities are added. Therefore there is no cancellation in (6.6.27), every arithmetic operation is done with high relative accuracy (see Section 2.5), and all quantities are computed to high relative precision. If the process is repeated to convergence, the resulting eigenvalue will be obtained to high relative accuracy, regardless of how tiny it is. For numerical confirmation try Exercise 6.6.61.

This all assumes that $\rho = 0$ at every step. Even if ρ is nonzero, we get the same outcome as long as ρ is small enough that $A - \rho I$ is positive definite. In that case it can be shown [55] that the s_j remain positive, so the addition in (6.6.27) is still that of two positive numbers. Some cancellation is inevitable when the shift is subtracted, but whatever is subtracted is no greater than what will be restored when the accumulated shift is added to a computed eigenvalue in the end. It follows (that is, it can be shown) that every eigenvalue is computed with high relative accuracy.

Thus we achieve high relative accuracy by using a shifting strategy that always underestimates the smallest eigenvalue among those that have not already been deflated out. The eigenvalues are found in order, from smallest to largest.

These conclusions should be tempered by the following observation. The eigenvalues that are computed accurately are those of a tridiagonal matrix. If this matrix was obtained by reduction of some full symmetric matrix T to tridiagonal form, there is no guarantee that we have computed the eigenvalues of T to high relative accuracy. The tridiagonal reduction algorithm (5.5.4) is normwise backward stable, but it does not preserve tiny eigenvalues of T to high relative accuracy.[5]

Historical Remark: Our order of development of this subject has been almost exactly the opposite of the historical order. Rutishauser introduced the qd algorithm (but not the differential form) in 1954 [57, 58] as a method for computing poles of meromorphic functions. A few years later he placed his parameters in matrices and introduced the more general LR algorithm [59]. This provided, in turn, the impetus for the development of the QR algorithm around 1960 [25, 45]. For many

[5]Of course, all eigenvalues *are* computed accurately relative to $\|T\|$, by which we mean that each approximation μ to an eigenvalue λ satisfies $|\mu - \lambda|/\|T\| \approx Cu$ for some C that isn't much bigger than one.

years the QR algorithm was seen as the pinnacle of these developments, and in some senses it is. However, by the early nineties Fernando and Parlett [22, 55] realized that differential versions of the qd algorithm could be used solve the symmetric tridiagonal eigenvalue problem (and the related singular value problem) with greater accuracy than the QR algorithm, thus giving new life to the qd algorithm.

Computing Eigenvectors by Inverse Iteration

Once we have calculated some eigenvalues by bisection or the dqds algorithm, if we want the corresponding eigenvectors, we can find them by inverse iteration, using the computed eigenvalues as shifts, as described in Section 5.3. For each eigenvalue λ_k this requires an LDL^T (or related) decomposition of $A - \lambda_k I$, which costs $O(n)$ flops, and one or two steps of inverse iteration, also costing $O(n)$ flops. Thus the procedure is quite economical.

In exact arithmetic, the eigenvectors of a symmetric matrix are orthogonal. A weakness of the inverse iteration approach is that it produces (approximate) eigenvectors that are not truly orthogonal. In particular, the computed eigenvectors associated with a tight cluster of eigenvalues can be far from orthogonal. One remedy is to apply the Gram-Schmidt process to these vectors to orthonormalize them. This costs $O(nm^2)$ work for a cluster of m eigenvalues, so it is satisfactory if m is not too large. However, for a matrix with huge clusters of eigenvalues, the added cost is unacceptable. A better remedy is to use *twisted factorizations*.

Computing Accurate Eigenvectors Using Twisted Factorizations

The super-accurate eigenvalues obtained by dqds can help us to compute super-accurate, numerically orthogonal eigenvectors, provided we perform inverse iteration in an appropriate way. Again it is important to work with factored forms (relatively robust representations) of the matrix rather than the matrix itself. Another key is to use special factorizations called *twisted factorizations*.

The *RRR algorithm*, an algorithm built using these factorizations, takes one step of inverse iteration, at a cost of $O(n)$ flops, for each eigenvector. Thus a complete set of eigenvalues and eigenvectors of a symmetric, tridiagonal matrix can be obtained in $O(n^2)$ flops. More generally, k eigenvalues and corresponding eigenvectors can be found in $O(nk)$ flops. Since each eigenvector is determined with very high accuracy, the computed eigenvectors are automatically orthogonal to working precision. Thus the potentially expensive Gram-Schmidt step mentioned above is avoided. The flop count $O(nk)$ is optimal in the following sense. Since k eigenvectors are determined by nk numbers in all, and at least one flop (not to mention fetching, storing, etc.) must be expended in the computation of each number, there is no way to avoid $O(nk)$ work.

An implementation of the RRR algorithm is included in LAPACK [1]. The details of the algorithm are complicated. Here we will just indicate some of the basic ideas.

We assume once again that we have a symmetric, tridiagonal matrix stored in the factored form LDL^T. We have already seen how to obtain that factored form of a shifted matrix

$$LDL^T - \rho I = \hat{L}\hat{D}\hat{L}^T \tag{6.6.28}$$

by (6.6.12). For our further development we need a companion factorization

$$LDL^T - \rho I = \check{U}\check{D}\check{U}^T, \tag{6.6.29}$$

where $\check{U}$ is unit upper bidiagonal:

$$\check{U} = \begin{bmatrix} 1 & \check{u}_1 & & & & \\ & 1 & \check{u}_2 & & & \\ & & 1 & \ddots & & \\ & & & \ddots & \check{u}_{n-2} & \\ & & & & 1 & \check{u}_{n-1} \\ & & & & & 1 \end{bmatrix}.$$

From Section 1.7 we know that LU decompositions, and hence also LDL^T decompositions such as (6.6.28), are natural byproducts of Gaussian elimination, performed in the usual top-to-bottom manner. Our preference for going from top to bottom and from left to right is simply a prejudice. If one prefers, one can start from the bottom, using the (n,n) entry as a pivot for the elimination of all of the entries above it in the nth column, and then move upward and to the left. If one performs Gaussian elimination in this way, one obtains as a byproduct a decomposition of the form UL, of which the UDU^T decomposition in (6.6.29) is a variant. Thus we expect to be able to compute the parameters $\check{d}_j$ and $\check{u}_j$ in (6.6.29) by a bottom-to-top process. Indeed, one easily checks (Exercise 6.6.62) that the following algorithm does the job.

$$\begin{array}{l} p_n \leftarrow d_n - \rho \\ \text{for } j = n, \ldots, 2 \\ \quad \left[\begin{array}{l} \check{d}_j \leftarrow p_j + d_{j-1} l_{j-1}^2 \\ q_{j-1} \leftarrow d_{j-1}/\check{d}_j \\ \check{u}_{j-1} \leftarrow l_{j-1} q_{j-1} \\ p_{j-1} \leftarrow p_j q_{j-1} - \rho \end{array} \right. \\ \check{d}_1 \leftarrow p_1 \end{array} \tag{6.6.30}$$

This is very similar to (6.6.27), the biggest difference being that it works from bottom to top.

Related to both (6.6.28) and (6.6.29) are the *twisted factorizations*. The kth twisted factorization has the form

$$LDL^T - \rho I = N_k D_k N_k^T, \tag{6.6.31}$$

where D_k is diagonal and N_k is "twisted," partly lower and partly upper triangular:

$$N_k = \begin{bmatrix} 1 & & & & & & & \\ \hat{l}_1 & 1 & & & & & & \\ & \ddots & \ddots & & & & & \\ & & \hat{l}_{k-1} & 1 & \check{u}_k & & & \\ & & & \ddots & \ddots & & & \\ & & & & & 1 & \check{u}_{n-1} & \\ & & & & & & 1 & \end{bmatrix}. \tag{6.6.32}$$

There are n twisted factorizations of $LDL^T - \rho I$, corresponding to $k = 1, \ldots, n$. The twisted factorizations for the cases $k = 1$ and $k = n$ are (6.6.29) and (6.6.28), respectively. It is easy to compute a twisted factorization. The entries $\hat{l}_1, \ldots, \hat{l}_{k-1}$ are easily seen to be the same as the $\hat{l}_j$ in (6.6.28). Likewise the entries $\check{u}_n, \ldots, \check{u}_k$ are the same as in (6.6.29). The diagonal matrix D_k has the form

$$\operatorname{diag}\left\{\hat{d}_1, \ldots, \hat{d}_{k-1}, \delta_k, \check{d}_{k+1}, \ldots, \check{d}_n\right\},$$

where $\hat{d}_1, \ldots, \hat{d}_{k-1}$ are from (6.6.28), and $\check{d}_{k+1}, \ldots, \check{d}_n$ are from (6.6.29). The only entry that cannot be grabbed directly from either (6.6.28) or (6.6.29) is δ_k, the "middle" entry of D_k. Checking the (k,k) entry of the equation (6.6.31), we find that

$$d_k + d_{k-1}l_{k-1}^2 - \rho = \hat{d}_{k-1}\hat{l}_{k-1}^2 + \delta_k + \check{d}_{k+1}\check{u}_k^2,$$

and therefore

$$\delta_k = d_k + d_{k-1}l_{k-1}^2 - \rho - \hat{d}_{k-1}\hat{l}_{k-1}^2 - \check{d}_{k+1}\check{u}_k^2.$$

Referring back to (6.6.12), (6.6.30), and Exercise 6.6.62, we find that δ_k can also be expressed as

$$\delta_k = s_k + p_{k+1}q_k. \tag{6.6.33}$$

This is a more robust formula.

Exercise 6.6.34 Check the assertions of the previous paragraph. □

We now see that we can compute all n twisted factorizations at once. We just need to compute (6.6.28) and (6.6.29) by algorithms (6.6.12) and (6.6.30), respectively, saving the auxiliary quantities s_j, p_j, and q_j. We use these to compute the δ_k in (6.6.33). This gives us all of the ingredients for all n twisted factorizations for $O(n)$ flops.

The RRR algorithm uses the twisted factorizations to compute eigenvectors associated with well-separated eigenvalues. Using a computed eigenvalue as the shift ρ, we compute the twisted factorizations $N_k D_k N_k^T$ simultaneously, as just described. In theory the D_k matrix in each factorization should should have a zero entry on the main diagonal, since $LDL^T - \rho I$ should be exactly singular if ρ is an eigenvalue.

Further inspection shows that precisely the entry δ_k should be zero. In practice, due to roundoff errors, none of the δ_k will be exactly zero. Although any of the twisted factorizations can be used to perform a step of inverse iteration, from a numerical standpoint it is best to use the one for which $|\delta_k| = \min_j |\delta_j|$. Specifically, we solve $N_k D_k N_k^T x = e_k \delta_k$, where e_k is the kth standard basis vector. The vector x, or $x/\|x\|_2$ if one wishes, is a very accurate eigenvector. The factor δ_k is included just for convenience. In fact, the computation is even easier than it looks. Exercise 6.6.63 shows that x satisfies

$$N_k^T x = e_k,$$

which can be solved by "outward substitution" as follows.

$$\begin{array}{l} x_k \leftarrow 1 \\ \text{for } j = k-1, \ldots, 1 \\ \quad [\ x_j \leftarrow -l_j x_{j+1} \\ \text{for } j = k, \ldots, n-1 \\ \quad [\ x_{j+1} \leftarrow -\breve{u}_j x_j \end{array} \tag{6.6.35}$$

This procedure requires $n-1$ multiplications and no additions or subtractions. Thus there are no cancellations, and x is computed to high relative accuracy. For more details and an error analysis see [17].

For clustered eigenvalues a more complex strategy is needed. Shift the matrix by ρ, where ρ is very near the cluster. Compute a new representation by Algorithm (6.6.12). This shifted matrix has many eigenvalues near zero. Although the absolute separation of these eigenvalues is no different than it was before the shift, their relative separation is now much greater, as they are all now much smaller in magnitude. Of course their relative accuracy is also much smaller than it was, so they need to be recomputed or refined. Once they have been recomputed to high relative accuracy, their eigenvectors can be computed using twisted factorizations, as described above. These highly accurate eigenvectors are numerically orthogonal to each other as well as to all of the other computed eigenvectors. This process must be repeated for each cluster. For details consult [16], [17], and [56].

Cuppen's Divide-and-Conquer Algorithm

As of this writing, the prevailing expert opinion is that Cuppen's algorithm will be supplanted by the dqds and RRR algorithms. We include here a brief description of this interesting scheme. The algorithm is implemented in LAPACK [1]. For more details see [12], [20].

A symmetric, tridiagonal matrix A can be rewritten as $A = \tilde{A} + H$, where

$$\tilde{A} = \begin{bmatrix} \tilde{A}_1 & 0 \\ 0 & \tilde{A}_2 \end{bmatrix}$$

$$= \left[\begin{array}{ccccc|ccccc} \alpha_1 & \beta_1 & & & & & & & & \\ \beta_1 & \alpha_2 & \ddots & & & & & & & \\ & \ddots & \ddots & \beta_{i-1} & & & & & & \\ & & \beta_{i-1} & (\alpha_i - \beta_i) & 0 & & & & & \\ \hline & & & 0 & (\alpha_i - \beta_i) & \beta_{i+1} & & & & \\ & & & & \beta_{i+1} & \alpha_{i+2} & \ddots & & & \\ & & & & & \ddots & \ddots & \beta_{n-1} & & \\ & & & & & & \beta_{n-1} & \alpha_n & & \end{array}\right]$$

and H is a very simple matrix of rank one whose only nonzero entries consist of the submatrix

$$\begin{bmatrix} \beta_i & \beta_i \\ \beta_i & \beta_i \end{bmatrix}$$

in the "middle" of H. This can be done for any choice of i, but our interest is in the case $i \approx n/2$, for which the submatrices $\tilde{A}_1$ and $\tilde{A}_2$ are of about the same size.

The eigenproblem for $\tilde{A}$ is easier than that for A, since it consists of two separate problems for $\tilde{A}_1$ and $\tilde{A}_2$, which can be solved independently, perhaps in parallel. Suppose we have the eigenvalues and eigenvectors of $\tilde{A}$. Can we deduce the eigensystem of $A = \tilde{A} + H$ by a simple updating procedure? As we shall see, the answer is yes. Cuppen's algorithm simply applies this idea recursively. Here is a thumbnail sketch of the algorithm: If A is 1×1, its eigenvalue and eigenvector are returned. Otherwise, A is decomposed to $\tilde{A} + H$ as shown above, and the algorithm computes the eigensystems of $\tilde{A}_1$ and $\tilde{A}_2$ by calling itself. This gives the eigensystem of $\tilde{A}$. Then the eigensystem of $A = \tilde{A}+H$ is obtained by the updating procedure (described below). This simple divide-and-conquer idea works very well. In practice we might prefer not to carry the subdivisions all the way down to the 1×1 level. Instead we can set some threshold size (e.g. 10×10) and compute eigensystems of matrices that are below that size by the QR algorithm or some other method.

Rank-One Updates

It remains to show how to obtain the eigenvalues and eigenvectors of A from those of $\tilde{A}$, where $A = \tilde{A} + H$ and H has rank one.

Exercise 6.6.36 Let $H \in \mathbb{R}^{n \times n}$ be a matrix of rank one.

(a) Show that there exist nonzero vectors $u, v \in \mathbb{R}^n$ such that $H = uv^T$.

(b) Show that if H is symmetric, then there exist nonzero $\rho \in \mathbb{R}$ and $w \in \mathbb{R}^n$ such that $H = \rho w w^T$. Show that w can be chosen so that $\|w\|_2 = 1$, in which case ρ is uniquely determined. To what extent is w nonunique?

(c) Let $H = \rho w w^T$, where $\|w\|_2 = 1$. Determine the complete eigensystem of H.

□

Now we can write $A = \tilde{A} + \rho w w^T$, where $\|w\|_2 = 1$. Since we have the entire eigensystem of $\tilde{A}$, we have $\tilde{A} = \tilde{Q}\tilde{D}\tilde{Q}^T$, where $\tilde{Q}$ is an orthogonal matrix whose columns are eigenvectors, and $\tilde{D}$ is a diagonal matrix whose main-diagonal entries are eigenvalues of $\tilde{A}$. Thus

$$A = \tilde{Q}(\tilde{D} + \rho z z^T)\tilde{Q}^T,$$

where $z = \tilde{Q}^T w$. We seek an orthogonal Q and diagonal D such that $A = QDQ^T$. If we can find an orthogonal $\hat{Q}$ and a diagonal D such that $\tilde{D} + \rho z z^T = \hat{Q}D\hat{Q}^T$, then $A = QDQ^T$, where $Q = \tilde{Q}\hat{Q}$. Thus it suffices to consider the matrix $\tilde{D} + \rho z z^T$.

The eigenvalues of $\tilde{D}$ are its main-diagonal entries, $\tilde{d}_1, \ldots, \tilde{d}_n$, and the associated eigenvectors are $e_1, \ldots, e_n$, the standard basis vectors of $\mathbb{R}^n$. The next exercise shows that it can happen that some of these are also eigenpairs of $\tilde{D} + \rho z z^T$.

Exercise 6.6.37 Let z_i denote the ith component of z. Suppose $z_i = 0$ for some i.

(a) Show that the ith column of $\tilde{D} + \rho z z^T$ is $\tilde{d}_i e_i$ and the ith row is $\tilde{d}_i e_i^T$.

(b) Show that $\tilde{d}_i$ is an eigenvalue of $\tilde{D} + \rho z z^T$ with associated eigenvector e_i.

(c) Show that if $q \in \mathbb{R}^n$ is an eigenvector of $\tilde{D} + \rho z z^T$ with associated eigenvalue $\lambda \neq \tilde{d}_i$, then q_i, the ith component of q, is zero.

□

From this exercise we see that if $z_i = 0$, we get an eigenpair for free. Furthermore the ith row and column of $\tilde{D} + \rho z z^T$ can be ignored during the computation of the other eigenpairs. thus we effectively work with a submatrix; that is, we deflate the problem.

Another deflation opportunity arises if two or more of the $\tilde{d}_i$ happen to be equal.

Exercise 6.6.38 Suppose $\tilde{d}_1 = \tilde{d}_2 = \cdots = \tilde{d}_k$. Construct a reflector U such that $U(\tilde{D} + \rho z z^T)U^T = \tilde{D} + \rho \tilde{z}\tilde{z}^T$, and $\tilde{z}$ has $k-1$ entries equal to zero. At what point are you using the fact that $\tilde{d}_1 = \cdots = \tilde{d}_k$? □

Thanks to Exercise 6.6.38, we see that whenever we have k equal $\tilde{d}_i$, we can deflate $k-1$ eigenpairs.

Thanks to these deflation procedures, we can now assume without loss of generality that all of the z_i are nonzero and the $\tilde{d}_i$ are all distinct. We may also assume, by reordering if necessary, that $\tilde{d}_1 < \tilde{d}_2 < \cdots < \tilde{d}_n$. It is not hard to see what the eigenpairs of $\tilde{D} + \rho z z^T$ must look like. Let λ be an eigenvalue with associated eigenvector q. Then $(\tilde{D} + \rho z z^T)q = \lambda q$; that is,

$$(\tilde{D} - \lambda I)q + \rho z(z^T q) = 0. \tag{6.6.39}$$

It is not hard to show that the scalar $z^T q$ is nonzero and that λ is distinct from $\tilde{d}_1, \tilde{d}_2, \ldots, \tilde{d}_n$.

Exercise 6.6.40

(a) Use (6.6.39) to show that if $z^T q = 0$, then $\lambda = \tilde{d}_i$ and q is a multiple of e_i for some i. Conclude that $z_i = 0$, in contradiction to one of our assumptions.

(b) Use the ith row of (6.6.39) to show that if $\lambda = \tilde{d}_i$, then either $z^T q = 0$ or $z_i = 0$. Thus $\lambda \neq \tilde{d}_i$.

□

Since $\lambda \neq \tilde{d}_i$ for all i, $(\tilde{D} - \lambda I)^{-1}$ exists. Multiplying equation (6.6.39) by $(\tilde{D} - \lambda I)^{-1}$, we obtain

$$q + \rho(\tilde{D} - \lambda I)^{-1} z(z^T q) = 0. \tag{6.6.41}$$

Multiplying this equation by z^T on the left and then dividing by the nonzero scalar $z^T q$, we see that

$$1 + \rho z^T (\tilde{D} - \lambda I)^{-1} z = 0. \tag{6.6.42}$$

Since $(\tilde{D} - \lambda I)^{-1}$ is a diagonal matrix, (6.6.42) can also be written as

$$1 + \rho \sum_{i=1}^{n} \frac{z_i^2}{\tilde{d}_i - \lambda} = 0. \tag{6.6.43}$$

Every eigenvalue of $\tilde{D} + \rho z z^T$ must satisfy (6.6.43), which is known as the *secular equation*. A great deal can be learned about the solutions of (6.6.43) by studying the function

$$f(\lambda) = 1 + \rho \sum_{i=1}^{n} \frac{z_i^2}{\tilde{d}_i - \lambda}. \tag{6.6.44}$$

This is a rational function with the n distinct poles $\tilde{d}_1, \tilde{d}_2, \ldots, \tilde{d}_n$.

Exercise 6.6.45 Let f be as in (6.6.44)

(a) Compute the derivative of f. Show that if $\rho > 0$ ($\rho < 0$), then f is increasing (resp. decreasing) on each subinterval on which it is defined.

(b) Sketch the graph of f for each of the cases $\rho > 0$ and $\rho < 0$. Show that the secular equation $f(\lambda) = 0$ has exactly one solution between each pair of poles of f.

(c) Let $d_1 < d_2 < \cdots < d_n$ denote the solutions of the secular equation. Show that if $\rho > 0$, then $\tilde{d}_i < d_i < \tilde{d}_{i+1}$ for $i = 1, \ldots, n-1$.

(d) Recall from Exercise 5.4.53 that the trace of a matrix is equal to the sum of its eigenvalues. Applying this fact to the matrix $\tilde{D} + \rho z z^T$, show that $d_n < \tilde{d}_n + \rho$ if $\rho > 0$. (For a more general result see Exercise 6.6.64.)

(e) Show that if $\rho < 0$, then $\tilde{d}_{i-1} < d_i < \tilde{d}_i$ for $i = 2, \ldots, n$.

(f) Using a trace argument as in part (d), show that if $\rho < 0$, then $\tilde{d}_1 + \rho < d_1$. (This is extended in Exercise 6.6.64)

□

Thanks to Exercise 6.6.45, we know that each of the $n-1$ intervals $(\tilde{d}_i, \tilde{d}_{i+1})$ contains exactly one eigenvalue of $\tilde{D} + \rho z z^T$, and the nth eigenvalue is located in either $(\tilde{d}_n, \tilde{d}_n + \rho)$ or $(\tilde{d}_1 + \rho, \tilde{d}_1)$, depending upon the sign of ρ. Using this information, we can solve the secular equation (6.6.43) numerically and thereby determine the eigenvalues of $\tilde{D} + \rho z z^T$ with as much accuracy as we please. For this we could employ the bisection method, which was introduced earlier in this section, for example. However, much faster methods are available. A great deal of effort went into the development of the solver that is used in the LAPACK [1] implementation of this algorithm.

Once the eigenvalues of $\tilde{D} + \rho z z^T$ have been found, the eigenvectors can be obtained using (6.6.41). For each eigenvalue λ, an associated eigenvector q is given by

$$q = c(\tilde{D} - \lambda I)^{-1} z,$$

where c is any nonzero scalar. Thus the components of q are given by

$$q_i = \frac{c z_i}{\tilde{d}_i - \lambda} \qquad i = 1, \ldots, n.$$

In this brief sketch we have ignored numerous details.

Additional Exercises

Exercise 6.6.46 In this exercise you will show that the matrix

$$A = \begin{bmatrix} a & b \\ b & d \end{bmatrix}$$

can always be diagonalized by a rotator

$$Q = \begin{bmatrix} c & -s \\ s & c \end{bmatrix},$$

a *Jacobi rotator*.

(a) Show that

$$Q^T A Q = \begin{bmatrix} c^2 a + s^2 d + 2csb & (c^2 - s^2)b + cs(d-a) \\ (c^2 - s^2)b + cs(d-a) & c^2 d + s^2 a - 2csb \end{bmatrix}.$$

(b) Show that if $a = d$, then $Q^T A Q$ can be made diagonal by taking $c = s = 1/\sqrt{2}$. Otherwise, letting

$$\hat{t} = \frac{2b}{a-d}, \tag{6.6.47}$$

show that Q^TAQ is diagonal if and only if

$$\hat{t} = \frac{2cs}{c^2 - s^2} = \tan 2\theta,$$

where $c = \cos\theta$ and $s = \sin\theta$. There is a unique $\theta \in (-\pi/4, \pi/4)$ for which $\hat{t} = \tan 2\theta$.

(c) Show that for any $\theta \in (-\pi/4, \pi/4)$,

$$\tan\theta = \frac{\sin 2\theta}{1 + \cos 2\theta},$$

$$\cos 2\theta = \frac{1}{\sqrt{1 + \tan^2 2\theta}}, \qquad \sin 2\theta = \cos 2\theta \tan 2\theta,$$

and

$$\tan\theta = \frac{\tan 2\theta}{1 + \sqrt{1 + \tan^2 2\theta}}.$$

Conclude that the tangent $t = \tan\theta$ of the angle that makes Q^TAQ diagonal is given by

$$t = \frac{\hat{t}}{1 + \sqrt{1 + \hat{t}^2}}, \tag{6.6.48}$$

where $\hat{t}$ is as in (6.6.47). Show that $c = \cos\theta$ and $s = \sin\theta$ can be obtained from t by

$$c = \frac{1}{\sqrt{1 + t^2}} \qquad \text{and} \qquad s = ct.$$

(d) There is a (very slight) chance of overflow in (6.6.47) if $a - b$ is tiny. This can be eliminated by working with the reciprocal whenever $|a - b| < |2b|$. Let

$$\hat{k} = \frac{a - d}{2b}.$$

Then $\hat{k} = \cot 2\theta$, where θ is the angle of the desired rotator. Show that $t = \tan\theta$ is given by

$$t = \frac{\operatorname{sign}(\hat{k})}{|\hat{k}| + \sqrt{1 + \hat{k}^2}}. \tag{6.6.49}$$

(Rewrite (6.6.48) in terms of $\hat{k} = 1/\hat{t}$.)

(e) Show that

$$Q^TAQ = \begin{bmatrix} a + tb & 0 \\ 0 & d - tb \end{bmatrix}.$$

□

Exercise 6.6.50 Use the formulas developed in Exercise 6.6.46 to diagonalize the matrices

$$\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 5 & 6 \\ 6 & 1 \end{bmatrix}.$$

Do each computation two ways, once using (6.6.48), and once using (6.6.49). Check your answers by computing the eigenvalues of the matrices by some other means. □

Exercise 6.6.51 Show that the search for the largest off-diagonal entry of an $n \times n$ matrix costs $O(n^2)$ work, while the cost of applying a Jacobi rotator is $O(n)$. Thus, for large n, the classical Jacobi method expends the vast majority of its effort on the searches. □

Exercise 6.6.52 Let $\hat{A} = Q^T A Q$, where Q is the Jacobi rotator that sets a_{ij} to zero. Let D and $\hat{D}$ be diagonal matrices, and let E and $\hat{E}$ be symmetric matrices with zeros on the main diagonal, uniquely determined by the equations $A = D + E$ and $\hat{A} = \hat{D} + \hat{E}$.

(a) Prove that

$$\|\hat{E}\|_F^2 = \|E\|_F^2 - 2a_{ij}^2 \qquad \text{and} \qquad \|\hat{D}\|_F^2 = \|D\|_F^2 + 2a_{ij}^2.$$

Thus the Jacobi rotator transfers weight $2a_{ij}^2$ from off-diagonal onto main diagonal entries.

(b) In the classical Jacobi method we always annihilate the off-diagonal entry of greatest magnitude. Prove that the largest off-diagonal entry always satisfies $a_{ij}^2 \geq \|E\|_F^2/N$, where $N = n(n-1)$. Infer that $\|\hat{E}\|_F^2 \leq (1 - 1/N)\|E\|_F^2$. Thus after m steps of the classical Jacobi method, $\|E\|_F^2$ will have been reduced at least by a factor of $(1 - 2/N)^m$. Thus the classical Jacobi method is guaranteed to converge, and the convergence is no worse than linear. Exercise 6.6.53 will show that the convergence is in fact quadratic.

□

Exercise 6.6.53 Let $A \in \mathbb{R}^{n \times n}$ be a symmetric matrix with distinct eigenvalues $\lambda_1, \ldots, \lambda_n$, and let $\delta = \min\{|\lambda_i - \lambda_j| \mid i \neq j\}$. We will continue to use the notation $A = D + E$ established in Exercise 6.6.52. Suppose that at the beginning of a cyclic Jacobi sweep, $\|E\|_F = \epsilon$, where ϵ is small compared with δ. then $|a_{ij}| \leq \epsilon$ for all $i \neq j$. The elements must stay this small since $\|E\|_F$ in nonincreasing. Suppose further that $\|E\|_F$ is small enough that the main-diagonal entries of A are fairly close to the eigenvalues.

(a) Show that each rotator generated during the sweep must satisfy $|s| \leq O(\epsilon)$ and $c \approx 1$.

(b) Using the result of part (a), show that once each a_{ik} is set to zero, subsequent rotations can make it no bigger than $O(\epsilon^2)$. Thus, at the end of a complete

sweep, every off-diagonal entry is $O(\epsilon^2)$. This means that the convergence is quadratic.

A closer analysis reveals that quadratic convergence occurs even when the eigenvalues are not distinct. □

Exercise 6.6.54

(a) Compute the matrix $\Delta^{-1}A\Delta$, where A is as in (6.6.4), and

$$\Delta = \text{diag}\{\delta_1, \ldots, \delta_n\}$$

with all $\delta_j \neq 0$. Show how to choose the δ_j so that $\Delta^{-1}A\Delta$ has the form (6.6.14). Notice that δ_1 (or any one of the δ_j) can be chosen arbitrarily, after which all of the other δ_j are uniquely determined.

(b) Show that any matrix of the form (6.6.15) with all γ_j positive is diagonally similar to a matrix of the form (6.6.4) with $\beta_j = \sqrt{\gamma_j}$, $j = 1, \ldots, n-1$.

(c) Check that the decomposition $\tilde{A} = \tilde{L}\tilde{R}$ given by (6.6.16) holds with $\tilde{r}_j = \alpha_j - \tilde{l}_{j-1}$, $j = 1, \ldots, n$ (if we define $\tilde{l}_0 = 0$), and $\tilde{l}_j = \gamma_j/\tilde{r}_j$, $j = 1, \ldots, n-1$.

□

Exercise 6.6.55 This exercise shows how to apply a shift to a tridiagonal matrix $\tilde{A}$ in factored form without explicitly forming $\tilde{A}$. Suppose we have a decomposition $\tilde{A} = \tilde{L}\tilde{R}$ of the form (6.6.16), and we want to obtain the decomposition of a shifted matrix: $\tilde{A} - \rho I = \hat{L}\hat{R}$.

(a) Show that the entries of $\hat{L}$ and $\hat{R}$ can be obtained directly from those of $\tilde{L}$ and $\tilde{R}$ by the algorithm

$$\begin{array}{l} \hat{r}_1 \leftarrow \tilde{r}_1 - \rho \\ \text{for } j = 1, \ldots, n-1 \\ \quad \left[\begin{array}{l} \hat{l}_j \leftarrow \tilde{l}_j \tilde{r}_j / \hat{r}_j \\ \hat{r}_{j+1} \leftarrow \tilde{r}_{j+1} + \tilde{l}_j - \hat{l}_j - \rho \end{array} \right. \end{array}$$

This is the *obvious* form of the algorithm.

(b) Introduce auxiliary quantities t_j by $t_1 = -\rho$ and $t_{j+1} = \tilde{l}_j - \hat{l}_j - \rho = \hat{r}_{j+1} - \tilde{r}_{j+1}$, j= 1, ..., $n-1$. Show that the algorithm from part (a) can be rewritten as (6.6.18), which is the *differential* form of the algorithm. It is more accurate than the version from part (a).

□

Exercise 6.6.56 In this exercise you will prove Sylvester's law of inertia. Let A, $B \in \mathbb{R}^{n\times n}$ be symmetric matrices.

(a) Show that the dimension of the null space of A is equal to the number of eigenvalues of A that are zero.

(b) Show that if B is congruent to A, then $\mathcal{N}(A)$ and $\mathcal{N}(B)$ have the same dimension. Deduce that $\zeta(A) = \zeta(B)$ and $\nu(A) + \pi(A) = \nu(B) + \pi(B)$.

(c) Show that if B is congruent to A, then there exist diagonal matrices D and E and a nonsingular matrix C such that $E = C^T DC$, the eigenvalues of A are the main-diagonal entries of D, and the eigenvalues of B are the main-diagonal entries of E.

(d) Continuing from part (c), the number of positive entries on the main diagonal of D and E are equal to $\pi(A)$ and $\pi(B)$, respectively, and we would like to show that these are the same. Let us assume that they are different and show that this leads to a contradiction. Without loss of generality, assume E has more positive entries than D does. Show that there is a nonzero vector x such that

$$\begin{aligned} x_i &= 0 \qquad \text{if } e_{ii} \leq 0 \\ (Cx)_i &= 0 \qquad \text{if } d_{ii} > 0. \end{aligned}$$

(*Hint:* These are homogeneous linear equations in n unknowns. How many equations are there?)

(e) Let x be the nonzero vector from part (d), and let $z = Cx$. Show that $x^T Ex > 0$, $z^T Dz \leq 0$, and $x^T Ex = z^T Dz$, which yields a contradiction.

(f) Show that $(\nu(A), \zeta(A), \pi(A)) = (\nu(B), \zeta(B), \pi(B))$.

□

Exercise 6.6.57 Show that if $A - \rho I = LR$ and $RL + \rho I = \hat{A}$, then $\hat{A} = L^{-1}AL$. Thus the LR iteration is a similarity transformation. □

Exercise 6.6.58 Let L and R be as in (6.6.16), let $\hat{A} = RL$ and $\hat{A} - \rho I = \hat{L}\hat{R}$, where $\hat{L}$ and $\hat{R}$ have the same general form as in (6.6.16).

(a) Compute $\hat{A}$, $\hat{L}$, and $\hat{R}$, and verify that the entries of the factors $\hat{L}$ and $\hat{R}$ can be obtained directly from those of L and R by the algorithm (6.6.26).

(b) Show that if $l_{n-1} = 0$, then r_n is an eigenvalue of A and $\hat{A}$.

□

Exercise 6.6.59 Let $s_j = r_j - \rho - \hat{l}_{j-1}$, $j = 1, \ldots, n$, where $\hat{l}_0 = 0$. Show that in algorithm (6.6.26)

(a) $s_j = \hat{r}_j - l_j$, $j = 1, \ldots, n-1$.

(b) $s_{j+1} = r_{j+1} - r_{j+1} l_j / \hat{r}_j - \rho = r_{j+1} s_j / \hat{r}_j - \rho$, $j = 1, \ldots, n-1$.

(c) Confirm that (6.6.26) and (6.6.27) are equivalent in exact arithmetic.

□

Exercise 6.6.60 Let A be a tridiagonal matrix of the form (6.6.15) with all $\gamma_j > 0$, and suppose A is diagonally similar to a positive definite matrix. Then A has a decomposition $A = LR$ of the form (6.6.16). Show that the quantities $r_1, \ldots, r_n$ in R are all positive (cf. Exercise 6.6.17). Then show that the entries $l_1, \ldots, l_{n-1}$ of L are positive as well. □

Exercise 6.6.61 Consider the 5×5 matrix

$$A = LR = \begin{bmatrix} 1 & & & & \\ m & 1 & & & \\ & m & 1 & & \\ & & m & 1 & \\ & & & m & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & & & \\ & 1 & 1 & & \\ & & 1 & 1 & \\ & & & 1 & 1 \\ & & & & 1 \end{bmatrix},$$

where $m = 123456$.

(a) Using MATLAB, multiply L by R to get A explicitly, then use the `eig` command to compute the eigenvalues of A. Notice that there are four large eigenvalues around 123456 and one tiny one. The normwise backward stability of the QR algorithm (which `eig` uses) guarantees that the four large eigenvalues are accurate. The fifth eigenvalue is surely near zero, but there is no guarantee that `eig` has computed it accurately in a relative sense.

(b) Prove that the determinant of A is 1. Recall from Exercise 5.4.54 that the determinant of a matrix is the product of its eigenvalues. Thus $\det(A) = \lambda_1\lambda_2\lambda_3\lambda_4\lambda_5$. Compute $\lambda_1\lambda_2\lambda_3\lambda_4\lambda_5$ using the eigenvalues you computed in part (a), and notice that this product is far from 1. This proves that λ_5, the tiny eigenvalue, has not been computed accurately; it is off by a factor of about a million. To get the correct value, use the equation $\lambda_1\lambda_2\lambda_3\lambda_4\lambda_5 = 1$ to solve for λ_5: $\lambda_5 = 1/(\lambda_1\lambda_2\lambda_3\lambda_4)$. Since the large eigenvalues are certainly accurate, and this formula for λ_5 involves only multiplications and divisions (no cancellations), it gives the correct value of λ_5 to 15 decimal places or so. Use `format long e` to view all digits of λ_5.

(c) Write a MATLAB program that implements the dqds algorithm (6.6.27). Apply the dqds algorithm with zero shift to the factors of A. If your program is correct, it will give the correct value of the tiny eigenvalue to about five decimal places after only one iteration. Perform a second iteration and get the correct tiny eigenvalue to sixteen decimal places accuracy.

□

Exercise 6.6.62 Show that the equation (6.6.29) implies the following relationships.

(a) $d_j + d_{j-1}l_{j-1}^2 - \rho = \check{d}_j + \check{d}_{j+1}\check{u}_j^2$ for $j = 1, \ldots, n$, where we set $d_0 = l_0 = \check{d}_{n+1} = \check{u}_n = 0$.

(b) $l_j d_j = \check{d}_{j+1}\check{u}_j$ for $j = 1, \ldots, n-1$.

(c) The relationships from parts (a) and (b) can be used to construct an algorithm for computing $\check{U}$ and $\check{D}$. However, a better *differential* form of the algorithm is obtained by introducing the auxiliary quantities

$$p_j = d_j - \check{d}_{j+1}\check{u}_j^2 - \rho,$$

with $p_n = d_n - \rho$. Show that $p_j = \check{d}_j - d_{j-1}l_{j-1}^2$ for $j = 2, \ldots, n$, and $p_j = p_{j+1}(d_j/\check{d}_{j+1}) - \rho$ for $j = 2, \ldots, n-1$.

(d) Verify that algorithm (6.6.30) produces the parameters that define $\check{U}$ and $\check{D}$.

□

Exercise 6.6.63 Consider a twisted factorization $N_k D_k N_k^T$, as in (6.6.31), with N_k as in (6.6.32).

(a) Devise an "inward substitution" algorithm to solve systems of the form $N_k z = w$ for z, where w is a given vector. Show that the algorithm is simplified drastically when $w = e_k$, the kth standard unit vector; in fact, $N_k e_k = e_k$, so $z = e_k$.

(b) Show that $N_k D_k N_k^T x = e_k \delta_k$ if and only if $N_k^T x = e_k$.

(c) Devise an algorithm to solve systems of the form $N_k^T x = y$ by "outward substitution."

(d) Show that when $y = e_k$, the algorithm from part (c) reduces to (6.6.35).

□

Exercise 6.6.64 This exercise extends the eigenvalue bounds of Exercise 6.6.45. Prove the following results using the equation $\mathrm{tr}(\tilde{D} + \rho z z^T) = \mathrm{tr}(\tilde{D}) + \rho \mathrm{tr}(z z^T)$ and the inequalities proved in Exercise 6.6.44.

(a) If $\rho > 0$, then $d_i < \tilde{d}_i + \rho$ for $i = 1, \ldots, n$.

(b) If $\rho < 0$, then $d_i > \tilde{d}_i + \rho$ for $i = 1, \ldots, n$.

(c) Regardless of the sign of ρ, there exist positive constants $c_1, \ldots, c_n$ such that $c_1 + \cdots + c_n = 1$ and $d_i = \tilde{d}_i + c_i \rho$ for $i = 1, \ldots, n$.

□

Exercise 6.6.65 Let $A = \begin{bmatrix} 2 & 2 \\ 2 & 5 \end{bmatrix}$.

(a) Write A in the form $\tilde{D} + \rho z z^T$, where $\tilde{D}$ is diagonal.

(b) Graph the function $f(\lambda)$ of (6.6.44) given by this choice of $\tilde{D}$, ρ, and z.

(c) Calculate the eigenvalues and eigenvectors of A by performing a rank-one update.

□

6.7 THE GENERALIZED EIGENVALUE PROBLEM

Numerous applications give rise to a more general form of eigenvalue problem

$$Av = \lambda Bv,$$

where $A, B \in \mathbb{C}^{n\times n}$. This problem has a rich theory, and numerous algorithms for solving it have been devised. This section provides only a brief overview.

Eigenvalue problems of this type can arise from systems of linear differential equations of the form

$$B\dot{x} = Ax - b. \tag{6.7.1}$$

These differ from those studied in Section 5.1 by the introduction of the coefficient matrix B in front of the derivative term. We can solve such problems by essentially the same methodology as in we used in Section 5.1. First we consider the homogeneous system

$$B\dot{x} = Ax. \tag{6.7.2}$$

As in Section 5.1, we seek solutions of the homogeneous problem with the simple form $x(t) = g(t)v$, where $g(t)$ is a nonzero scalar-valued function of t, and v is a nonzero vector that is independent of time. Substituting this form into (6.7.2), we find that

$$\frac{\dot{g}}{g}Bv = Av,$$

which implies that $\dot{g}/g$ must be constant. Call the constant λ. Then $\dot{g} = \lambda g$ (which implies $g(t) = ce^{\lambda t}$), and

$$Av = \lambda Bv.$$

This is the generalized eigenvalue problem. Each solution (λ, v) yields a solution $e^{\lambda t}v$ of (6.7.2). Thus we must solve the generalized eigenvalue problem in order to solve the homogeneous problem (6.7.2). Apart from that, the solution procedure is the same as for the problems we discussed in Section 5.1.

Example 6.7.3 The electrical circuit in Figure 6.2 differs from the ones we looked at in Section 5.1 in that it has an inductor in the interior wire, which is shared by two loop currents. As before, we can obtain a system of differential equations for this circuit by applying Kirchhoff's voltage law: the sum of the voltage drops around each loop is zero. For the first loop, the voltage drops across the 1Ω and 3Ω resistors are $1x_1$ and $3x_1$ volts, respectively. The voltage drop across the 2 H inductor is $2\dot{x}_1$ volts. The drop across the 5Ω resistor is $5(x_1 - x_2)$ volts, since the net upward current in the middle wire is $x_1 - x_2$ amps. Similarly, the drop across the 4 H inductor is $4(\dot{x}_1 - \dot{x}_2)$ volts. Thus the equation for the first loop is

$$x_1 + 3x_1 + 2\dot{x}_1 + 5(x_1 - x_2) + 4(\dot{x}_1 - \dot{x}_2) = 0$$

or

$$6\dot{x}_1 - 4\dot{x}_2 = -9x_1 + 5x_2.$$

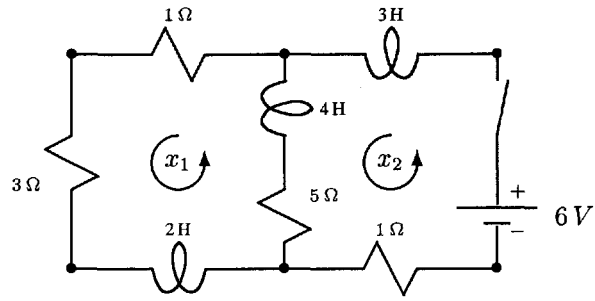

Fig. 6.2 Circuit with inductor shared by two loop currents

Similarly, the equation for the second loop is

$$-4\dot{x}_1 + 7\dot{x}_2 = 5x_1 - 6x_2 + 6.$$

Combining the two equations into a single matrix equation, we obtain

$$\begin{bmatrix} 6 & -4 \\ -4 & 7 \end{bmatrix} \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -9 & 5 \\ 5 & -6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 0 \\ -6 \end{bmatrix}. \tag{6.7.4}$$

which is a system of differential equations of the form $B\dot{x} = Ax - b$, as in (6.7.1).[6]

As a first step toward solving this system, we find a single solution z of (6.7.4). The simplest solution is a steady-state solution, which can be found by setting the derivative terms in (6.7.4) to zero. Doing so, we obtain the linear system

$$\begin{bmatrix} -9 & 5 \\ 5 & -6 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} 0 \\ -6 \end{bmatrix}.$$

Since the coefficient matrix is nonsingular, there is a unique steady-state solution, which we can determine by solving the system, either by pencil and paper or using MATLAB. Using MATLAB, we find that

$$z = \begin{bmatrix} 1.0345 \\ 1.8621 \end{bmatrix} \tag{6.7.5}$$

amperes.

The next step is to find the general solution of the homogeneous problem

$$\begin{bmatrix} 6 & -4 \\ -4 & 7 \end{bmatrix} \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -9 & 5 \\ 5 & -6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

[6]The matrices have additional interesting structure. Both A and B are symmetric, B is positive definite and A is negative definite. Alternatively, we can flip some signs and write the system as $B\dot{x} + Ax = b$, in which both B and A are positive definite.

As we have seen above, this requires solving the generalized eigenvalue problem

$$\begin{bmatrix} 6 & -4 \\ -4 & 7 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \lambda \begin{bmatrix} -9 & 5 \\ 5 & -6 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}.$$

This is small enough that we could solve it by pencil and paper, but we will use MATLAB instead. To get MATLAB's `eig` function to solve the generalized eigenvalue problem $Av = \lambda Bv$, we simply type `[V,D] = eig(A,B)`. When invoked in this way, `eig` uses the QZ algorithm, a generalization of the QR algorithm, which we will discuss later in this section. The output from `[V,D] = eig(A,B)` is a matrix V whose columns are (if possible) linearly independent eigenvectors and a diagonal matrix D whose main diagonal entries are the associated eigenvalues. Together they satisfy $AV = BVD$. As you can easily check, this means that $Av_j = \lambda_j Bv_j$, $j = 1, \ldots, n$, where v_j denotes the jth column of V, and λ_j is the (j, j) entry of D. Using `eig` in this way to solve our generalized eigenvalue problem, we obtain the eigenvalues

$$\lambda_1 = -1.5493 \qquad \text{and} \qquad \lambda_2 = -0.7199,$$

and associated eigenvectors

$$v_1 = \begin{bmatrix} 0.9708 \\ 0.2399 \end{bmatrix} \qquad \text{and} \qquad v_2 = \begin{bmatrix} 0.4126 \\ 0.9109 \end{bmatrix}.$$

Since these vectors are linearly independent, the general solution of the homogeneous problem is

$$c_1 e^{-1.5493t} \begin{bmatrix} 0.9708 \\ 0.2399 \end{bmatrix} + c_2 e^{-0.7199t} \begin{bmatrix} 0.4126 \\ 0.9109 \end{bmatrix},$$

where c_1 and c_2 are arbitrary constants. We now obtain the general solution to the nonhomogeneous problem by adding in the steady-state solution (6.7.5):

$$x(t) = \begin{bmatrix} 1.0345 \\ 1.8621 \end{bmatrix} + c_1 e^{-1.5493t} \begin{bmatrix} 0.9708 \\ 0.2399 \end{bmatrix} + c_2 e^{-0.7199t} \begin{bmatrix} 0.4126 \\ 0.9109 \end{bmatrix}. \quad (6.7.6)$$

Notice that both of the exponential functions decay as t increases. This is so because both eigenvalues are negative. Therefore, whatever the loop currents may initially be they will (quickly) settle down to their steady-state values. The system is stable.

Now suppose the switch in the circuit is closed at time zero. Then the subsequent values of the loop currents are given by the unique solution of the system that satisfy the initial conditions

$$x(0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Setting $t = 0$ in (6.7.6) and inserting these initial values, we obtain the linear system

$$\begin{bmatrix} 0.9708 & 0.4126 \\ 0.2399 & 0.9109 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} -1.0345 \\ -1.8621 \end{bmatrix}.$$

Doing so, we obtain $c_1 = -0.2215$ and $c_2 = -1.9859$.

A similar example is considered in Exercise 6.7.42. □

Basic Properties of the Generalized Eigenvalue Problem

Consider the generalized eigenvalue problem.

$$Av = \lambda Bv. \tag{6.7.7}$$

This is a generalization of the standard eigenvalue problem, reducing to standard problem in the case $B = I$. A nonzero $v \in \mathbb{C}^n$ that satisfies (6.7.7) for some value of λ is called an *eigenvector* of the ordered pair (A, B), and λ is called the associated *eigenvalue*. Clearly (6.7.7) holds if and only if $(\lambda B - A)v = 0$, so λ is an eigenvalue of (A, B) if and only if the *characteristic equation*

$$\det(\lambda B - A) = 0$$

is satisfied. One easily checks that $\det(\lambda B - A)$ is a polynomial of degree n or less. It is called the *characteristic polynomial* of the pair (A, B). The expression $\lambda B - A$ is often called a *matrix pencil*, and one speaks of the eigenvalues, eigenvectors, characteristic equation, and so forth, of the matrix pencil.

Exercise 6.7.8 Verify that $\det(\lambda B - A)$ is a polynomial, and its degree is at most n. □

Most generalized eigenvalue problems can be reduced to standard eigenvalue problems. For example, if B is nonsingular, the eigenvalues of the pair (A, B) are just the eigenvalues of $B^{-1}A$.

Exercise 6.7.9 Suppose B is nonsingular.

(a) Show that v is an eigenvector of (A, B) with associated eigenvalue λ if and only if v is an eigenvector of $B^{-1}A$ with eigenvalue λ.

(b) Show that v is an eigenvector of (A, B) with associated eigenvalue λ if and only if Bv is an eigenvector of AB^{-1} with eigenvalue λ.

(c) Show that $B^{-1}A$ and AB^{-1} are similar matrices.

(d) Show that the characteristic equation of (A, B) is essentially the same as that of $B^{-1}A$ and AB^{-1}.

□

From Exercise 6.7.9 we see that if B is nonsingular, the pair (A, B) must have exactly n eigenvalues, counting multiplicity. If B is singular, the situation is different. Consider the following example.

Example 6.7.10 Let

$$A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},$$

and notice that B is singular. You can easily verify that the characteristic equation of (A, B) is $\lambda - 1 = 0$, whose degree is less than the order of the matrices. Evidently

the pair (A, B) has only one eigenvalue, namely $\lambda = 1$. As we will see in a moment, it is reasonable to attribute to (A, B) a second eigenvalue $\lambda = \infty$. □

Suppose the pair (A, B) has a nonzero eigenvalue λ. Then, making the substitution $\mu = 1/\lambda$ in (6.7.7) and multiplying through by μ, we obtain

$$Bv = \mu Av,$$

which is the generalized eigenvalue problem for the ordered pair (B, A). We conclude that the nonzero eigenvalues of (B, A) are the reciprocals of the nonzero eigenvalues of (A, B). Now suppose B is singular. Then zero is an eigenvalue of (B, A). In light of the reciprocal relationship we have just noted, it is reasonable to regard ∞ as an eigenvalue of (A, B). We will do just that. With this convention, most generalized eigenvalue problems have n eigenvalues. There is, however, a class of problems that is not so well behaved. Let's look at an example.

Example 6.7.11 Let

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Then $\det(\lambda B - A) = 0$ for all λ, so every complex number is an eigenvalue. □

Exercise 6.7.12

(a) Let (A, B) be as in Example 6.7.11. Find an eigenvector associated with each eigenvalue. Find the nullspaces $\mathcal{N}(A)$ and $\mathcal{N}(B)$.

(b) Let A and B be any two matrices for which $\mathcal{N}(A) \cap \mathcal{N}(B) \neq \{0\}$. Show that every complex number is an eigenvalue of the pair (A, B), and every nonzero $v \in \mathcal{N}(A) \cap \mathcal{N}(B)$ is an eigenvector of (A, B) associated with every λ.

□

Any pair (A, B) for which the characteristic polynomial $\det(\lambda B - A)$ is identically zero is said to be *singular*. The term *singular pencil* is frequently used. If (A, B) is not singular, it is said to be *regular*. Certainly (A, B) is regular whenever A or B is a nonsingular matrix. As we have just seen, (A, B) is singular whenever $\mathcal{N}(A)$ and $\mathcal{N}(B)$ have a nontrivial intersection. It can also happen that a pair is singular even if $\mathcal{N}(A) \cap \mathcal{N}(B) = \{0\}$.

Example 6.7.13 Let

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

Then the pair (A, B) is singular, as $\det(\lambda B - A) = 0$ for all λ. Both A and B are singular matrices; however, $\mathcal{N}(A) \cap \mathcal{N}(B) = \{0\}$. □

Exercise 6.7.14 Verify the assertions of Example 6.7.13. □

It can also happen that a pair (A, B) is regular, even though both A and B are singular matrices.

Example 6.7.15 Let

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Then A and B are singular, but (A, B) is a regular pair □

Exercise 6.7.16 Let A and B be as in Example 6.7.15.

(a) Show that both A and B are singular matrices.

(b) Calculate the characteristic polynomial of the pair (A, B), and conclude that (A, B) is regular. What are the eigenvalues of (A, B)?

□

For the rest of this section we will restrict our attention to regular pencils. For more information on singular pencils see [13, 75, 82] and Volume II of [29].

In most applications at least one of A and B is nonsingular. Whenever this is the case, it is possible to reduce the generalized eigenvalue problem to a standard eigenvalue problem for one of the matrices AB^{-1}, $B^{-1}A$, BA^{-1}, or $A^{-1}B$. Although this is a possibility that should not be ruled out, there are a number of reasons why it might not be the best course of action. Suppose we compute AB^{-1}. If B is ill conditioned (in the sense of Chapter 2), the eigenvalues of the computed AB^{-1} can be poor approximations of the eigenvalues of (A, B), even though some or all of the eigenvalues of (A, B) are well conditioned (Exercise 6.7.43). A second reason not to compute AB^{-1} is that A and B might be symmetric. In many applications it is desirable to preserve the symmetry, and AB^{-1} will typically not be symmetric. Finally, A and B might be sparse, in which case we would certainly like to exploit the sparseness. But AB^{-1} will not be sparse, for the inverse of a sparse matrix is typically not sparse. For these reasons it is useful to develop algorithms that solve the generalized eigenvalue problem directly.

Equivalence Transformations

We know that two matrices have the same eigenvalues if they are similar, a fact which is exploited in numerous algorithms. If we wish to develop algorithms in the same spirit for the generalized problem, we need to know what sorts of transformations we can perform on pairs of matrices without altering the eigenvalues. It turns out that an even larger class than similarity transformations works. Two pairs of matrices (A, B) and $(\tilde{A}, \tilde{B})$ are said to be *equivalent* if there exist nonsingular matrices U and V such that $\tilde{A} = UAV$ and $\tilde{B} = UBV$. We can write the relationship more briefly

as $\tilde{A} - \lambda\tilde{B} = U(A - \lambda B)V$. In the next exercise you will show that equivalent pairs have the same eigenvalues.

Exercise 6.7.17 Suppose (A, B) and $(\tilde{A}, \tilde{B})$ are equivalent pairs: $\tilde{A} - \lambda\tilde{B} = U(A - \lambda B)V$.

(a) Show that λ is an eigenvalue of (A, B) with associated eigenvector v if and only if λ is an eigenvalue of $(\tilde{A}, \tilde{B})$ with associated eigenvector $V^{-1}v$.

(b) Show that (A, B) and $(\tilde{A}, \tilde{B})$ have essentially the same characteristic equation.

□

Exercise 6.7.18 Investigate the equivalence transformations given by each of the following pairs of nonsingular matrices, assuming that the indicated inverses exist: (a) $U = B^{-1}$, $V = I$, (b) $U = I$, $V = B^{-1}$, (c) $U = A^{-1}$, $V = I$, (d) $U = I$, $V = A^{-1}$. □

Computing Generalized Eigensystems: The Symmetric Case

If $A, B \in \mathbb{R}^{n\times n}$ are symmetric matrices, and B is positive definite, the pair (A, B) is called a *symmetric* pair. Given a symmetric pair (A, B), B has a Cholesky decomposition $B = R^T R$, where R is upper triangular. The equivalence transformation given by $U = R^{-T}$ and $V = R^{-1}$ transforms the generalized eigenvalue problem $Av = \lambda Bv$ to the standard eigenvalue problem $R^{-T}AR^{-1}y = \lambda y$, where $y = Rv$. Notice that the coefficient matrix $\tilde{A} = R^{-T}AR^{-1}$ is symmetric. This proves that a symmetric pair has real eigenvalues, and it has n linearly independent eigenvectors. The latter are not orthogonal in the conventional sense, but they are orthogonal with respect to an appropriately chosen inner product.

Exercise 6.7.19 The symmetric matrix $\tilde{A} = R^{-T}AR^{-1}$ has orthonormal eigenvectors $v_1, \ldots, v_n$, so $R^{-1}v_1, \ldots, R^{-1}v_n$ are eigenvectors of (A, B). Prove that these are orthonormal with respect to the *energy* inner product $\langle x, y\rangle_B = y^T Bx$. □

It is interesting that the requirement that B be positive definite is essential; it is not enough simply to specify that A and B be symmetric. It can be shown (cf. [54, p. 304]) that every $C \in \mathbb{R}^{n\times n}$ can be expressed as a product $C = AB^{-1}$, where A and B are symmetric. This means that the eigenvalue problem for any C can be reformulated as a generalized eigenvalue problem $Av = \lambda Bv$, where A and B are symmetric. Thus one easily finds examples of pairs (A, B) for which A and B are symmetric but the eigenvalues are complex.

One way to solve the eigenvalue problem for the symmetric pair (A, B) is to perform the transformation to $R^{-T}AR^{-1}$ and use one of the many available techniques for solving the symmetric eigenvalue problem. This has two of the same drawbacks as the transformation to AB^{-1}: 1.) If B is ill conditioned, the eigenvalues will not be determined accurately, and 2.) $R^{-T}AR^{-1}$ does not inherit any sparseness that A and B might possess. The second drawback is not insurmountable. If B is sparse, its Cholesky factor R will usually also be somewhat sparse. For example, if B is banded,

R is also banded. Because of this it is possible to apply sparse eigenvalue methods such as the Lanczos algorithm (6.3.23) to $\tilde{A} = R^{-T}AR^{-1}$ at a reasonable cost. The only way in which this method uses $\tilde{A}$ is to compute vectors $\tilde{A}x$ for a sequence of choices of x. For $\tilde{A}$ of the given form we can compute $\tilde{A}x$ by first solving $Ry = x$ for y, then computing $z = Ay$, then solving $R^T w = z$ for w. Clearly $w = \tilde{A}x$. If R is sparse, the two systems $Ry = x$ and $R^T w = z$ can be solved cheaply.

In many applications both A and B are positive definite. If A is much better conditioned than B, it might make sense to interchange the roles of A and B and take a Cholesky decomposition of A.

There are numerous other approaches to solving the symmetric generalized eigenvalue problem. A number of methods for the symmetric standard eigenvalue problem can be adapted to the generalized problem. For example, both the Jacobi method and the bisection method, described in Section 6.6, have been adapted to the generalized problem. See [54] for details and references.

Exercise 6.7.20 If B is symmetric and positive definite, then it has a spectral decomposition $B = VDV^T$ where V is orthogonal and D is diagonal and positive definite. Show how this decomposition can be used to reduce the symmetric generalized eigenvalue problem to a standard symmetric problem. □

Exercise 6.7.21 Two pairs (A, B) and $(\tilde{A}, \tilde{B})$ are said to be *congruent* if there exists a nonsingular $X \in \mathbb{R}^{n\times n}$ such that $\tilde{A} = XAX^T$ and $\tilde{B} = XBX^T$. Clearly any two congruent pairs are equivalent, but not conversely. Show that if (A, B) is a symmetric pair (and in particular B is positive definite), then $(\tilde{A}, \tilde{B})$ is also a symmetric pair. Thus we can retain symmetry by using congruence transformations. The transformation to a standard eigenvalue problem using the Cholesky factor and the transformation suggested in Exercise 6.7.20 are both examples of congruence transformations. □

Computing Generalized Eigensystems: The General Problem

We now consider the problem of finding the eigenvalues of a pair (A, B) that is not symmetric. We will develop an analogue of the reduction to upper Hessenberg form and an extension of the QR algorithm. Unfortunately these algorithms do not preserve symmetry.

Schur's Theorem 5.4.11 guarantees that every matrix in $\mathbb{C}^{n\times n}$ is unitarily similar to an upper triangular matrix. The analogous result for the generalized eigenvalue problem is

Theorem 6.7.22 *(Generalized Schur Theorem) Let $A, B \in \mathbb{C}^{n\times n}$. Then there exist unitary $Q, Z \in \mathbb{C}^{n\times n}$ and upper triangular $T, S \in \mathbb{C}^{n\times n}$ such that $Q^*AZ = T$ and $Q^*BZ = S$. Thus*

$$Q^*(A - \lambda B)Z = T - \lambda S.$$

A proof is worked out in Exercise 6.7.44.

The pair (T, S) is equivalent to (A, B), so it has the same eigenvalues. But the eigenvalues of (T, S) are evident because

$$\det(\lambda S - T) = (\lambda s_{11} - t_{11})(\lambda s_{22} - t_{22}) \cdots (\lambda s_{nn} - t_{nn}).$$

If there is a k for which both s_{kk} and t_{kk} are zero, the characteristic polynomial is identically zero; that is, the pair is singular. Otherwise the eigenvalues are t_{kk}/s_{kk}, $i = 1, \ldots, n$. Some of these quotients may be ∞.

The numbers s_{kk} and t_{kk} give more information than the ratio s_{kk}/t_{kk}. If both s_{kk} and t_{kk} are near zero, the pair (A, B) is nearly singular in the sense that it is close to a singular pair.

Exercise 6.7.23 Use Theorem 6.7.22 to verify the following facts. Suppose (A, B) is a regular pair. Then the degree of the characteristic polynomial is less than or equal to n. It is exactly n if and only if B is nonsingular. If B is singular, then the pair (A, B) has at least one infinite eigenvalue. The degree of the characteristic polynomial is equal to n minus the number of infinite eigenvalues. □

For real (A, B) there is a real decomposition analogous to the Wintner-Murnaghan theorem (Theorem 5.4.22). In this variant, Q and Z are real, orthogonal matrices, S is upper triangular, and T is quasitriangular.

Reduction to Hessenberg-Triangular Form

Theorem 6.7.22 and its real analogue give us simple forms to shoot for. Perhaps we can devise an algorithm that produces a sequence of equivalent pencils that converges to one of those forms. A big step in that direction is to transform the pair (A, B) to the Hessenberg-Triangular form described in the next theorem.

Theorem 6.7.24 *Let $A, B \in \mathbb{C}^{n\times n}$. Then there exist unitary $Q, Z \in \mathbb{C}^{n\times n}$, upper Hessenberg $H \in \mathbb{C}^{n\times n}$ and upper triangular $U \in \mathbb{C}^{n\times n}$ such that*

$$Q^*(A - \lambda B)Z = H - \lambda U.$$

There is a stable, direct algorithm to calculate Q, Z, H, and U in $O(n^3)$ flops. If A and B are real, Q, Z, H, and U can be taken to be real.

Proof. The proof will consist of an outline of an algorithm for transforming (A, B) to (H, U). This transformation will be effected by a sequence of reflectors and rotators applied on the left and right of A and B. Each transformation applied on the left (right) makes a contribution to Q^* (resp. Z). Every transformation applied to A must also be applied to B and vice versa. We know from Chapter 3 that B can be reduced to upper triangular form by a sequence of reflectors applied on the left. Applying this sequence of reflectors to both B and A, we obtain a new (A, B), for which B is upper triangular. The rest of the algorithm consists of reducing A to upper Hessenberg form without destroying the upper triangular form of B. The first major step is to introduce zeros in the first column of A. This is done one element at a

time, from bottom to top. First the $(n,1)$ entry is set to zero by a rotator (or reflector) acting in the $(n,n-1)$ plane, applied to A on the left. This alters rows $n-1$ and n of A. The same transformation must also be applied to B. You can easily check that this operation will introduce one nonzero entry in the lower triangle of B, in the $(n,n-1)$ position. This blemish can be removed by the applying the appropriate rotator to columns $n-1$ and n of B. That is, we apply a rotation in the $(n,n-1)$ plane to B on the right. Applying the same rotation to A, we do not disturb the zero that we previously introduced in column 1. We next apply a rotation to rows $n-2$ and $n-1$ of A in such a way that a zero is produced in position $(n-1,1)$. Applying the same rotation to B, we introduce a nonzero in position $(n-1,n-2)$. This can be removed by applying a rotator to columns $n-1$ and $n-2$. This operation does not disturb the zeros in the first column of A. Continuing in this manner we can produce zeros in positions $(n-2,1)$, $(n-3,1)$, ..., $(3,1)$. This scheme cannot be used to produce a zero in position $(2,1)$. In order to do that we would have to apply a rotator to rows 1 and 2 of A. The same rotator applied to B would produce a nonzero in position $(2,1)$. We could eliminate that entry by applying a rotator to columns 1 and 2 of B. Applying the same rotator to columns 1 and 2 of A, we would destroy all of the zeros that we had so painstakingly created in column 1. Thus we must leave the $(2,1)$ entry of A as it is and move on to column 2. The second major step is to introduce zeros in column 2 in positions $(n,2)$, $(n-1,2)$, ..., $(4,2)$ by the same scheme as we used to create zeros in column 1. You can easily check that none of the rotators that are applied on the right operate on either column 1 or 2, so the zeros that have been created so far are maintained. We then proceed to column 3, and so on. After $n-2$ major steps, we are done. □

Exercise 6.7.25 Count the flops required to execute the algorithm outlined in the proof of Theorem 6.7.24, (a) assuming Q and Z are not to be assembled, (b) assuming Q and Z are to be assembled. In either case the count is $O(n^3)$. □

The QZ Algorithm

We can now restrict our attention to pairs (A,B) for which A is upper Hessenberg and B is upper triangular. We can even assume that A is properly upper Hessenberg, since otherwise the eigenvalue problem can be reduced to subproblems in the obvious way. We will also assume that B is nonsingular, although this assumption can be lifted. A form of the implicit QR algorithm called the QZ algorithm [51] can be used to find the eigenvalues of such pairs.[7] Our development will parallel that of the QR algorithm for the SVD in Section 5.9.

Each step of the QZ algorithm will effectively perform a step of the QR algorithm on AB^{-1} and $B^{-1}A$ simultaneously. Ultimately we want to do this without forming

[7]When MATLAB's `eig` function is called upon to solve a generalized eigenvalue problem, it reduces the pair to Hessenberg-triangular form, as described above, then completes the reduction to generalized Schur form by the QZ algorithm.

these products or inverting B. Let us begin, however, with a naive version of the algorithm. If we want to perform QR steps with shift ρ, we begin by taking the QR decompositions

$$AB^{-1} - \rho I = \hat{Q}R \qquad \text{and} \qquad B^{-1}A - \rho I = \hat{Z}S, \tag{6.7.26}$$

where $\hat{Q}$ and $\hat{Z}$ are unitary, and R and S are upper triangular. If we now define $\hat{A}$ and $\hat{B}$ by

$$\hat{A} = \hat{Q}^{-1}A\hat{Z} \qquad \text{and} \qquad \hat{B} = \hat{Q}^{-1}B\hat{Z}, \tag{6.7.27}$$

we see immediately that

$$\hat{A}\hat{B}^{-1} = \hat{Q}^{-1}AB^{-1}\hat{Q} = R\hat{Q} + \rho I \tag{6.7.28}$$

and

$$\hat{B}^{-1}\hat{A} = \hat{Z}^{-1}B^{-1}A\hat{Z} = S\hat{Z} + \rho I. \tag{6.7.29}$$

Thus the transformations $AB^{-1} \to \hat{A}\hat{B}^{-1}$ and $B^{-1}A \to \hat{B}^{-1}\hat{A}$ are both shifted QR steps.

If we can perform the transformation (6.7.27), we will have accomplished the QR steps implicitly, so that is now our objective. For this we need the matrices $\hat{Q}$ and $\hat{Z}$, and we would like to obtain them without performing the QR decompositions (6.7.26) or even forming the matrices AB^{-1} and $B^{-1}A$. Each of $\hat{Q}$ and $\hat{Z}$ is a product of rotators:

$$\hat{Q} = \hat{Q}_1 \cdots \hat{Q}_{n-1} \qquad \text{and} \qquad \hat{Z} = \hat{Z}_1 \cdots \hat{Z}_{n-1}. \tag{6.7.30}$$

If we have these rotators, we can apply them to (A, B), one after the other, to transform (A, B) to $(\hat{A}, \hat{B})$. The challenge is to find an inexpensive way to generate each rotator as it is needed, so that we can effect (6.7.27) efficiently. In Section 5.7 we saw how to do this for the standard eigenvalue problem. We could build a similar argument here, but instead we will simply describe the implicit QZ algorithm, and then use the implicit-Q theorem to justify it.

Before we describe the algorithm, we will demonstrate that the QZ transformation (6.7.27) preserves the Hessenberg-triangular form of the pair. If ρ is not an eigenvalue of the pair (A, B), then the triangular matrices R and S in (6.7.26) are nonsingular.

Exercise 6.7.31 Suppose R and S in (6.7.26) are nonsingular.

(a) Show that

$$A(B^{-1}A - \rho I) = (AB^{-1} - \rho I)A$$

and

$$B(B^{-1}A - \rho I) = (AB^{-1} - \rho I)B.$$

(b) Show that $\hat{A} = RAS^{-1}$ and $\hat{B} = RBS^{-1}$.

(c) Show that if B is upper triangular, then so is $\hat{B}$.

(d) Show that if A is (properly) upper Hessenberg, then so is $\hat{A}$.

□

These results all hold if ρ is not an eigenvalue. Even when ρ is an eigenvalue, they remain mostly true. For example, $\hat{A}$ would be a properly upper Hessenberg matrix, except that $\hat{a}_{n,n-1} = 0$ (exposing an eigenvalue). We will skip the discussion of this case, which requires a bit more effort.

Now we can describe the implicit QZ step. To get started, we need the first column of $\hat{Q}$. From the equation $AB^{-1} - \rho I = \hat{Q}R$ of (6.7.26) and the fact that R is upper triangular, we see that the first column of $\hat{Q}$ is proportional to the first column of $AB^{-1} - \rho I$, which is easily seen to be

$$x = \begin{bmatrix} a_{11}b_{11}^{-1} - \rho \\ a_{21}b_{11}^{-1} \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{6.7.32}$$

We can compute x with negligible effort. Let Q_1 be a rotator (or reflector) in the $(1,2)$ plane whose first column is proportional to x. Notice that x_2 is certainly nonzero, which implies that Q_1 is a nontrivial rotator. Transform (A, B) to (Q_1^*A, Q_1^*B). This transformation recombines the first two rows of each matrix. It does not disturb the upper Hessenberg form of A, but it does disturb the upper triangular form of B, creating a bulge in the $(2,1)$ position. (Draw a picture.) Because $b_{11} \neq 0$ and Q_1 is a nontrivial rotator, this bulge is certainly nonzero.

The rest of the algorithm consists of returning the pair to Hessenberg-triangular form by chasing the bulge. Let Z_1 be a (nontrivial) rotator acting on columns 1 and 2 that eliminates the bulge; that is, for which $Q_1^*BZ_1$ is again upper triangular. Applying the same transformation to A we obtain $Q_1^*AZ_1$, which has a bulge in the $(3,1)$ position. Because $a_{32} \neq 0$ and Z_1 is a nontrivial rotator, this new bulge is certainly nonzero. It can be eliminated by left multiplication by a nontrivial rotator Q_2^* acting in the $(2,3)$ plane. This operation deposits a nonzero entry in the $(2,1)$ position of $Q_2^*Q_1^*AZ_1$. This entry will not be altered by subsequent transformations. Applying Q_2^* to B as well, we find that $Q_2^*Q_1^*BZ_1$ has a bulge (which is certainly nonzero) in the $(3,2)$ position. We have now chased the bulge from the $(2,1)$ position of B over to A and back to the $(3,2)$ position of B. We can now chase it to the $(4,2)$ position of A by applying a rotator Z_2 on the right, and so on. The pattern is now clear. The bulge can be chased the rest of the way down the A and B matrices. It is finally chased away completely by Z_{n-1}, which acts on columns $n-1$ and n to remove the bulge from the $(n, n-1)$ position of the transformed B matrix without introducing a bulge in A. This completes the implicit QZ step. We have transformed (A, B) to $(\tilde{A}, \tilde{B})$, given by

$$\tilde{A} = \tilde{Q}^*A\tilde{Z} \qquad \tilde{B} = \tilde{Q}^*B\tilde{Z}, \tag{6.7.33}$$

where

$$\tilde{Q} = Q_1Q_2\cdots Q_{n-1} \quad \text{and} \quad \tilde{Z} = Z_1Z_2\cdots Z_{n-1}. \tag{6.7.34}$$

$\tilde{Q}$ and $\tilde{Z}$ are unitary matrices. $\tilde{A}$ is upper Hessenberg, and $\tilde{B}$ is upper triangular.

Exercise 6.7.35 Verify that the subdiagonal entries of $\tilde{A}$ satisfy $\tilde{a}_{k+1,k} \neq 0$ for $k = 1, 2, \ldots, n-2$. (You might also like to show that $\tilde{a}_{n,n-1} \neq 0$ if and only if the shift ρ is not an eigenvalue of (A, B).) □

Exercise 6.7.36 Verify that the flop count for an implicit QZ step, including accumulation of the transforming matrices Q and Z, is $O(n^2)$. □

We wish to show that the pair $(\tilde{A}, \tilde{B})$ is essentially the same as the pair $(\hat{A}, \hat{B})$ given by (6.7.27).[8] The following theorem makes this possible.

Theorem 6.7.37 *Let $A \in \mathbb{C}^{n\times n}$, and let $B \in \mathbb{C}^{n\times n}$ be nonsingular. Suppose $\tilde{B}$ and $\hat{B}$ are upper triangular and nonsingular, $\tilde{A}$ is upper Hessenberg, $\hat{A}$ is properly upper Hessenberg, $\tilde{Q}$, $\hat{Q}$, $\tilde{Z}$, $\hat{Z}$ are unitary,*

$$\tilde{A} = \tilde{Q}^{-1} A \tilde{Z}, \qquad \tilde{B} = \tilde{Q}^{-1} B \tilde{Z}, \tag{6.7.38}$$

$$\hat{A} = \hat{Q}^{-1} A \hat{Z}, \qquad \hat{B} = \hat{Q}^{-1} B \hat{Z}. \tag{6.7.39}$$

Suppose further that $\tilde{Q}$ and $\hat{Q}$ have essentially the same first columns; that is, $\hat{q}_1 = \tilde{q}_1 d_1$, where $|d_1| = 1$. Then there exist unitary, diagonal matrices D and E such that $\hat{Q} = \tilde{Q}D$, $\hat{Z} = \tilde{Z}E$,

$$\hat{A} = D^{-1}\tilde{A}E \qquad \text{and} \qquad \hat{B} = D^{-1}\tilde{B}E.$$

In other words, $(\hat{A}, \hat{B})$ is essentially the same as $(\tilde{A}, \tilde{B})$.

Proof. The matrix $\tilde{A}\tilde{B}^{-1} = \tilde{Q}^{-1}AB^{-1}\tilde{Q}$ is upper Hessenberg, and $\hat{A}\hat{B}^{-1} = \hat{Q}^{-1}AB^{-1}\hat{Q}$ is properly upper Hessenberg. Since $\tilde{Q}$ and $\hat{Q}$ have essentially the same first column, we can apply the implicit-Q theorem (Theorem 5.7.24) with AB^{-1}, $\hat{A}\hat{B}^{-1}$, and $\tilde{A}\tilde{B}^{-1}$, playing the roles of A, $\hat{A}$, and $\tilde{A}$, respectively, to conclude that $\hat{Q} = \tilde{Q}D$ for some unitary, diagonal D. Rewriting the "B" parts of (6.7.38) and (6.7.39) as

$$\tilde{Z}\tilde{B}^{-1} = B^{-1}\tilde{Q} \qquad \text{and} \qquad \hat{Z}\hat{B}^{-1} = B^{-1}\hat{Q},$$

and using $\hat{Q} = \tilde{Q}D$, we have

$$\hat{Z}\hat{B}^{-1} = B^{-1}\hat{Q} = B^{-1}\tilde{Q}D = \tilde{Z}(\tilde{B}^{-1}D).$$

Defining $F = \hat{Z}\hat{B}^{-1} = \tilde{Z}(\tilde{B}^{-1}D)$, we see that $\hat{Z}\hat{B}^{-1}$ and $\tilde{Z}(\tilde{B}^{-1}D)$ are both QR decompositions of F. Therefore, by essential uniqueness of QR decompositions (Exercise 3.2.60), $\tilde{Z}$ and $\hat{Z}$ are essentially the same; that is, there is a unitary diagonal E such that $\hat{Z} = \tilde{Z}E$. The other assertions follow. □

[8] Furthermore, the rotators $Q_1, \ldots, Q_{n-1}$ and $Z_1, \ldots, Z_{n-1}$ are (essentially) the same as the rotators $\hat{Q}_1, \ldots, \hat{Q}_{n-1}$ and $\hat{Z}_1, \ldots, \hat{Z}_{n-1}$, respectively (6.7.30), but we will not prove this.

To apply the theorem, note that the transforming matrix $\tilde{Q} = Q_1 \cdots Q_{n-1}$ of (6.7.34) has same first column as Q_1, since each of the rotators $Q_2, \ldots, Q_{n-1}$ acts on columns other than the first. But we chose Q_1 so that its first column is proportional to that of $\hat{Q}$ of (6.7.26) and (6.7.27), so $\tilde{Q}$ and $\hat{Q}$ have essentially the same first column. We can now apply Theorem 6.7.37 to conclude that the pair $(\tilde{A}, \tilde{B})$ of (6.7.33) is essentially the same as the pair $(\hat{A}, \hat{B})$ of (6.7.27). Recalling (6.7.28) and (6.7.29), we see that our implicit QZ step effects QR iterations on both AB^{-1} and $B^{-1}A$. This completes the justification of the implicit QZ step, at least in the case when the shift is not an eigenvalue.

Now suppose we perform a sequence of QZ steps to generate a sequence of equivalent pairs (A_k, B_k). Then if the shifts are chosen reasonably, the matrices $C_k = A_k B_k^{-1}$ and $E_k = B_k^{-1} A_k$ will converge to upper triangular or quasitriangular form. Thus $A_k = C_k B_k = B_k E_k$ will also converge to quasitriangular form, from which the generalized eigenvalues are either evident or can be determined easily.

Another question that needs to be addressed is that of choice of shifts. From what we know about the QR algorithm, we know that it makes sense to take the shift for the kth step to be an eigenvalue of the lower-right-hand 2-by-2 submatrix of either $A_{k-1}B_{k-1}^{-1}$ or $B_{k-1}^{-1}A_{k-1}$. Exercise 6.7.46 shows how to determine these submatrices without determining B_{k-1}^{-1} explicitly.

Our development has assumed that B is nonsingular. However, the implicit QZ algorithm works just as well if B is singular. In this case, the pair (A, B) has an infinite eigenvalue (assuming the pair is regular). There is at least one zero on the main diagonal of B, which is moved steadily upward by the QZ algorithm until an infinite eigenvalue can be deflated at the top [78]. Another approach is to deflate out all the infinite eigenvalues before applying the QZ algorithm. This approach is developed in Exercise 6.7.47.

We have derived a single-step QZ algorithm. It is also natural to consider a double-step QZ algorithm analogous to the double-step QR algorithm, to be used in order to avoid complex arithmetic when A and B are real. Such an algorithm is indeed possible. It is just like the single-step algorithm, except that the bulges are fatter.

Exercise 6.7.40 Derive a double-step implicit QZ algorithm. □

Additional Exercises

Exercise 6.7.41 Using MATLAB, check that the calculations in Example 6.7.3 are correct. Plot the solutions for $0 \leq t \leq 5$. (You may find it helpful to review Exercises 5.1.19 and 5.1.20.) □

Exercise 6.7.42 In working this exercise, do not overlook the advice in Exercises 5.1.19 and 5.1.20. In Figure 6.3 all of the resistances are 1 Ω and all of the inductances are 1 H, except for the three that are marked otherwise. Initially the loop currents are all zero, because the switch is open. Suppose the switch is closed at time 0.

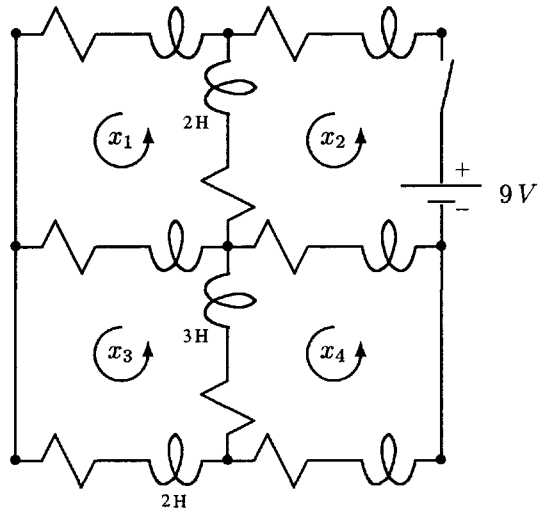

Fig. 6.3 Solve for the time-varying loop currents.

(a) Write down a system of four differential equations for the four unknown loop currents. Write your system in the matrix form $B\dot{x} = Ax - b$.

(b) Solve the system $Az = b$ to obtain a steady-state solution of the differential equation.

(c) Get the eigenvalues and eigenvectors of A, and use them to construct the general solution of the homogeneous equation $B\dot{z} = Az$.

(d) Deduce the solution of the initial value problem $B\dot{x} = Ax - b$, $x(0) = 0$.

(e) Plot the loop currents $x_1(t), \ldots, x_4(t)$ for $0 \leq t \leq 6$ seconds. (Checkpoint: Do these look realistic?)

□

Exercise 6.7.43 Perform the following experiment using MATLAB. Build two matrices A and B as follows:

```
n = 4;
M = randn(n);
A = M'*M
cond(A)
N = randn(n);
B = N'*N
lam = eig(B);
shift = min(lam) - 10*eps;
```

```
B = B - shift*eye(n);
cond(B)
```

These are random positive definite matrices. On average A should be reasonably well conditioned. However, B has been modified to make it extremely ill conditioned.

(a) Explain why B should be ill conditioned.

(b) Compute the eigenvalues of the pair (A, B) two different ways:

```
format long e
lg = eig(A,B);
lb = eig(A*inv(B));
```

Either way you get one huge eigenvalue and $n-1$ smaller eigenvalues. Compare the two results. Probably they will not agree very well.

(c) Decide which eigenvalues are better by computing some eigenvectors (by `[V,D] = eig(A,B)`, e.g.) and computing each residual norm $\| Av - \lambda Bv \|$ twice, once using the `lg` "eigenvalue" and once using the `lb` "eigenvalue."

□

Exercise 6.7.44 In this exercise you will prove the Generalized Schur theorem, which states that every pair (A, B) is unitarily equivalent to an upper-triangular pair (T, S): $Q^*(A - \lambda B)Z = T - \lambda S$. The proof is by induction on n, the dimension of the matrices.

(a) Confirm that Theorem 6.7.22 is true when $n = 1$.

(b) Now suppose we have a problem of size $n > 1$. We need to reduce it to one of size $n-1$ so that we can use induction. The key is to produce a pair of vectors v and $w \in \mathbb{C}^n$ such that $\| v \|_2 = 1$, $\| w \|_2 = 1$,

$$Av = \alpha w \quad \text{and} \quad Bv = \beta w, \tag{6.7.45}$$

where α and β are complex scalars. Suppose first that B is nonsingular. Then the pair (A, B) has n eigenvalues. Let μ be any one of the eigenvalues, and let v be a corresponding eigenvector such that $\| v \|_2 = 1$. Define w to be an appropriate multiple of Av, and show that (6.7.45) is satisfied for appropriate choices of α and β satisfying $\mu = \alpha/\beta$.

(c) Now suppose B is singular. Show that if we choose v so that $Bv = 0$, then (6.7.45) is satisfied for appropriate choices of w, α, and β. (Consider two cases: $Av \neq 0$ (infinite eigenvalue) and $Av = 0$ (singular pencil).)

(d) We have now established that in all cases we can find v and w with unit norm, such that (6.7.45) holds. Let V and W be unitary matrices whose first columns

are v and w, respectively. Show that W^*AV and W^*BV are both block triangular; specifically,

$$W^*(\lambda B - A)V = \lambda \left[\begin{array}{c|ccc} \beta & * & \cdots & * \\ \hline 0 & & & \\ \vdots & & \hat{B} & \\ 0 & & & \end{array}\right] - \left[\begin{array}{c|ccc} \alpha & * & \cdots & * \\ \hline 0 & & & \\ \vdots & & \hat{A} & \\ 0 & & & \end{array}\right].$$

(e) Prove Theorem 6.7.22 by induction on n.

□

Exercise 6.7.46

(a) Suppose B is a nonsingular block triangular matrix

$$B = \begin{bmatrix} B_{11} & B_{12} \\ 0 & B_{22} \end{bmatrix}.$$

Show that B^{-1} has the block triangular form

$$B^{-1} = \begin{bmatrix} B_{11}^{-1} & X \\ 0 & B_{22}^{-1} \end{bmatrix}.$$

Determine what X must be.

(b) Let B be nonsingular and upper triangular. Show how to calculate the lower-right-hand k-by-k submatrix of B^{-1} cheaply for $k = 1, 2, 3$. Give explicit formulas.

(c) Let (A, B) have Hessenberg-triangular form. Let M denote the lower-right-hand 2-by-2 submatrix of $B^{-1}A$. By making the appropriate partition of $B^{-1}A$, show that M depends only on the lower-right-hand 2-by-2 submatrix of B^{-1}. Derive explicit formulas for the entries of M.

(d) Let (A, B) have Hessenberg-triangular form. Let N denote the lower-right-hand 2-by-2 submatrix of AB^{-1}. Show that N depends only on the lower-right-hand 3-by-3 submatrix of B^{-1}.

□

Exercise 6.7.47 Consider a pair (A, B), where A is upper Hessenberg, B is upper triangular, and $b_{jj} = 0$ for some j. Show that an infinite eigenvalue can be deflated from the pencil after a finite sequence of rotations.

(a) First outline an algorithm that deflates the infinite eigenvalue at the top of the pencil: The first rotator acts on B from the right and sets $b_{j-1,j-1}$ to zero. When this rotator is applied to A, it creates a bulge, which can then be removed

by a rotator applied on the left. This rotator must also be applied to B. Next another rotator is applied to B on the right to set $b_{j-2,j-2}$ to zero. This creates another bulge in A, which can be annihilated by a rotator acting on the left. Show that we can continue this way until the entry b_{11} has been set to zero, the modified B is upper triangular (and $b_{22} = 0$ also), and A is upper Hessenberg. Apply one more rotator on the left to A to set a_{21} to zero. Now show that an infinite eigenvalue can be deflated at the top.

(b) Outline an algorithm similar to the one in part (a) that chases the infinite eigenvalue to the bottom.

If $j \leq n/2$, the algorithm of part (a) is cheaper; otherwise the algorithm of part (b) is the more economical. □

This page intentionally left blank

7

Iterative Methods for Linear Systems

In this chapter we return to the problem of solving a linear system $Ax = b$, where A is $n \times n$ and nonsingular. This problem can be solved without difficulty, even for fairly large values of n, by Gaussian elimination on today's computers. However, once n becomes very large (e.g. several thousand) and the matrix A becomes very sparse (e.g. 99.9% of its entries are zeros), iterative methods become more efficient. This chapter begins with a section that shows how such large, sparse problems can arise. Then the classical iterative methods are introduced and analyzed. From there we move on to a discussion of descent methods, including the powerful conjugate gradient method for solving positive definite systems. The important idea of preconditioning is introduced along the way. The conjugate gradient method is just one of a large family of Krylov subspace methods. The chapter concludes with a brief discussion of Krylov subspace methods for indefinite and nonsymmetric problems.

We restrict our attention to real systems throughout the chapter. However, virtually everything said here can be extended to the complex case.

7.1 A MODEL PROBLEM

Large sparse matrices arise routinely in the numerical solution of partial differential equations (PDE). We will proceed by stages, beginning with a simple ordinary differential equation (ODE). This is a *one-dimensional* problem, in the sense that there is one independent variable, x. Suppose a function $f(x)$ is given for $0 \leq x \leq 1$, and we wish to find a function $u(x)$ satisfying the ODE

$$-u''(x) = f(x) \qquad 0 < x < 1 \tag{7.1.1}$$

and the boundary conditions

$$u(0) = T_0 \qquad \text{and} \qquad u(1) = T_1. \tag{7.1.2}$$

Here T_0 and T_1 are two given numbers. Boundary value problems of this type arise in numerous settings. For example, if we wish to determine the temperature distribution $u(x)$ inside a uniform wall, we must solve such a problem. The numbers T_0 and T_1 represent the ambient temperatures on the two sides of the wall, and the function $f(x)$ represents a heat source within the wall.

This is a very simple problem. Depending on the nature of f, you may be able to solve it easily by integrating f a couple of times and using the boundary conditions to solve for the constants of integration. Let us not pursue this approach; our plan is to use this problem as a stepping stone to a harder boundary value problem involving a PDE. Instead, let us pursue an approximate solution by the finite difference method, as we did in Example 1.2.12.

Let $h = 1/m$ be a small increment, where m is some large integer (e.g. 100 or 1000), and mark off equally spaced points x_i on the interval $[0, 1]$. Thus $x_0 = 0$, $x_1 = h$, $x_2 = 2h, \ldots, x_m = mh = 1$, or briefly $x_i = ih$ for $i = 0, 1, 2, \ldots, m$. We will approximate the solution of (7.1.1) on $x_0, x_1, \ldots, x_m$. Since (7.1.1) holds at each of these points, we have

$$-u''(x_i) = f(x_i) \qquad i = 1, \ldots, m-1.$$

As we noted in Example 1.2.12 (and Exercise 1.2.21), a good approximation for the second derivative is

$$u''(x_i) \approx \frac{u(x_{i-1}) - 2u(x_i) + u(x_{i+1})}{h^2}. \tag{7.1.3}$$

Substituting this approximation into the ODE, we obtain, for $i = 1, \ldots, m-1$,

$$\frac{-u(x_{i-1}) + 2u(x_i) - u(x_{i+1})}{h^2} \approx f(x_i).$$

These are approximations, not equations, but it is not unreasonable to expect that if we treat them as equations and solve them exactly, we will get good approximations to the true solution at the mesh points x_i. Thus we let $u_1, \ldots, u_{m-1}$ denote approximations to $u(x_1), \ldots, u(x_{m-1})$ obtained by solving the equations exactly:

$$-u_{i-1} + 2u_i - u_{i+1} = h^2 f(x_i), \qquad i = 1, \ldots, m-1. \tag{7.1.4}$$

We have multiplied through by h^2 for convenience. Since $x_0 = 0$ and $x_m = 1$, it makes sense to define $u_0 = T_0$ and $u_m = T_1$, in agreement with the boundary conditions (7.1.2). The boundary value u_0 is used in (7.1.4) when $i = 1$, and u_m is used when $i = m-1$. Equation (7.1.4) is actually a system of $m-1$ linear equations for the $m-1$ unknowns $u_1, \ldots, u_{m-1}$, so it can be written as a matrix equation

$$Au = b,$$

where u is now a column vector containing the unknowns,

$$A = \begin{bmatrix} 2 & -1 & & & & \\ -1 & 2 & -1 & & & \\ & -1 & 2 & \ddots & & \\ & & \ddots & \ddots & -1 & \\ & & & -1 & 2 & -1 \\ & & & & -1 & 2 \end{bmatrix}, \tag{7.1.5}$$

and

$$b = \begin{bmatrix} h^2 f(x_1) + T_0 \\ h^2 f(x_2) \\ h^2 f(x_3) \\ \vdots \\ h^2 f(x_{m-2}) \\ h^2 f(x_{m-1}) + T_1 \end{bmatrix}.$$

Notice that the boundary values end up in b, since they are known quantities. The coefficient matrix A is nonsingular (Exercise 7.1.14), so the system has a unique solution. Once we have solved it, we have approximations to $u(x)$ at the grid points x_i.

The matrix A has numerous useful properties. It is obviously symmetric. It is even positive definite (Exercise 7.1.15 or Exercise 7.3.37). It is also extremely sparse, tridiagonal, in fact. The reason for this is clear: each unknown is directly connected only to its nearest neighbors.

Such a simple system does not require iterative methods for its solution. This is the simplest case of a banded matrix. The band is preserved by the Cholesky decomposition, and the system can be solved in $O(m)$ flops. If Cholesky's method works, then so does Gaussian elimination without pivoting, which also preserves the tridiagonal structure. This variant is actually preferable, as it avoids calculating $m-1$ square roots. But now let us get to our main task, which is to generate examples of large sparse systems for which the use of iterative methods is advantageous.

A Model PDE (Two-Dimensional)

Let Ω denote a bounded region in the plane with boundary $\partial\Omega$, let $f(x, y)$ be a given function defined on Ω, and let $g(x, y)$ be defined on $\partial\Omega$. Consider the problem of finding a function $u(x, y)$ satisfying the partial differential equation

$$-\frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2} = f \quad \text{in } \Omega \tag{7.1.6}$$

subject to the boundary condition

$$u = g \quad \text{on } \partial\Omega. \tag{7.1.7}$$

The PDE (7.1.6) is called Poisson's equation. It can be used to model a broad range of phenomena, including heat flow, chemical diffusion, fluid flow, elasticity, and electrostatic potential. In a typical heat flow problem $u(x, y)$ represents the temperature at point (x, y) in a homogeneous post or pillar whose cross-section is Ω. The function g represents the specified temperature on the surface of the post, and $f(x, y)$ represents a heat source within the post. Since there are two independent variables, x and y, we call this a two-dimensional problem.

For certain special choices of f and g, the exact solution can be found, but usually we have to settle for an approximation. Let us consider how we might solve (7.1.6) numerically by the finite difference method. To minimize complications, we restrict our attention to the case where $\Omega = [0, 1] \times [0, 1]$, the unit square. Let $h = 1/m$ be an increment, and lay out a computational grid of points in Ω with spacing h. The (i, j)th grid point is $(x_i, y_j) = (ih, jh)$, with $i, j = 0, \ldots, m$. A grid with $h = 1/5$ is shown in Figure 7.1.

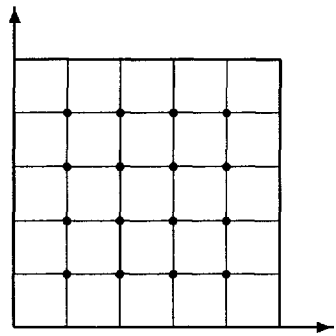

Fig. 7.1 Computational grid with $h = 1/5$ on $\Omega = [0, 1] \times [0, 1]$

We will approximate the PDE (7.1.6) by a system of difference equations defined on the grid. The approximation that we used for the ordinary derivative $u''(x)$ can also be used for the partial derivatives, since a partial derivative is just an ordinary derivative taken with respect to one variable while the other variable is held fixed. Thus the approximation (7.1.3) yields the approximation

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j) \approx \frac{u(x_{i-1}, y_j) - 2u(x_i, y_j) + u(x_{i+1}, y_j)}{h^2}.$$

The y variable is held fixed. Similarly we have for the y-derivative

$$\frac{\partial^2 u}{\partial y^2}(x_i, y_j) \approx \frac{u(x_i, y_{j-1}) - 2u(x_i, y_j) + u(x_i, y_{j+1})}{h^2}.$$

Substituting these approximations into (7.1.6), we find that a solution of (7.1.6) satisfies the approximation

$$\frac{-u(x_{i-1}, y_j) + 2u(x_i, y_j) - u(x_{i+1}, y_j)}{h^2} + \frac{-u(x_i, y_{j-1}) + 2u(x_i, y_j) - u(x_i, y_{j+1})}{h^2} \approx f(x_i, y_j)$$

at the interior grid points $i, j = 1, \ldots, m-1$. These are approximations, not equations, but, again, if we treat them as equations and solve them exactly, we should get a good approximation of the true solution $u(x, y)$. Consider, therefore, the system of equations

$$\frac{-u_{i-1,j} + 2u_{i,j} - u_{i+1,j}}{h^2} + \frac{-u_{i,j-1} + 2u_{i,j} - u_{i,j+1}}{h^2} = f_{i,j},$$

which becomes, after minor rearrangement,

$$-u_{i-1,j} - u_{i,j-1} + 4u_{i,j} - u_{i,j+1} - u_{i+1,j} = h^2 f_{i,j}, \qquad i, j = 1, \ldots, m-1. \tag{7.1.8}$$

The shorthand $f_{i,j} = f(x_i, y_j)$ has been introduced.

Each equation involves five of the unknown values, whose relative location in the grid is shown in the left-hand diagram of Figure 7.2. The weights with which the

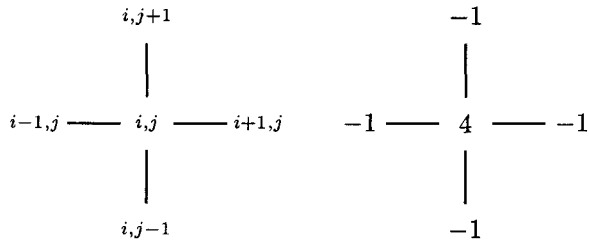

Fig. 7.2 Five-point stencil

five unknowns are combined are shown in the right-and diagram of Figure 7.2. This is called the *five-point stencil* for approximating Poisson's equation.

Boundary values of $u_{i,j}$, which occur in the equations for mesh points that are adjacent to the boundary, can be determined by the boundary conditions. For example, equations for the mesh points $(i, m-1)$ (adjacent to the top boundary) contain the "unknown" $u_{i,m}$, which can be specified by the boundary condition $u_{i,m} = g(x_i, 1)$. With this understanding, (7.1.8) can be seen to be a system of $(m-1)^2$ equations in the $(m-1)^2$ unknowns $u_{i,j}$, $i, j = 1, \ldots, m-1$. If we can solve these equations, we will have approximations $u_{i,j} \approx u(x_i, y_j)$ to the solution of (7.1.6) with boundary conditions (7.1.7) at the grid points.

The equations are linear, so they can be written as a matrix equation $Au = b$. Before we can do this, we need to decide on the order in which the unknowns $u_{i,j}$ should be packed into a single vector u. (The ordering was obvious in the ODE case.) There is one unknown for each grid point (Figure 7.1). We can order the $u_{i,j}$ by sweeping through the grid by rows, by columns, or by diagonals, for example. Let us do it by rows. The first row contains $u_{1,1}, \ldots, u_{m-1,1}$, the second row contains $u_{1,2}, \ldots, u_{m-1,2}$, and so on. Thus u will be the column vector defined by

$$u^T = [u_{1,1}, \ldots, u_{m-1,1}, u_{1,2}, \ldots, u_{m-1,2}, \ldots, u_{m-1,m-1}].$$